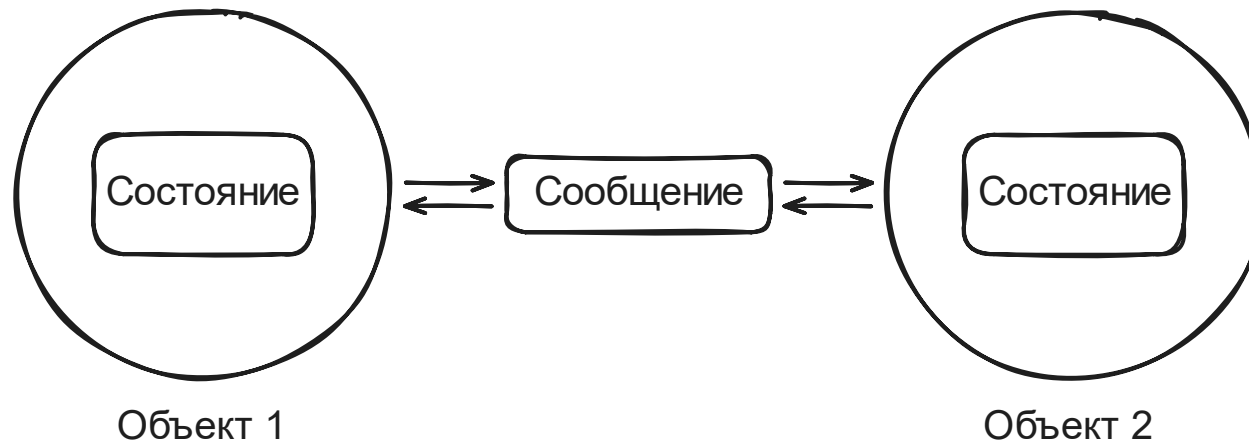


Алгоритмизация и программирование

Лекция 8 (C++)

Объектно-ориентированная модель

Система состоит из полностью изолированных друг от друга объектов взаимодействующих только при помощи сообщений.

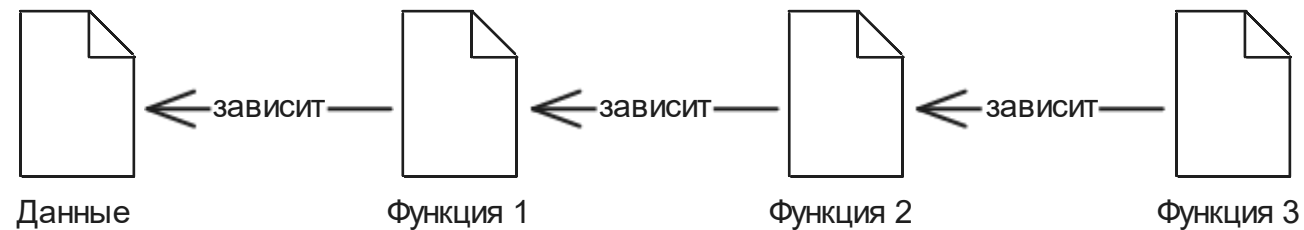


Классическая модель (сильная связность)

Классическая модель: программа – это набор данных и функций, которые эти данные преобразуют. Вычисление – это применение функции к данным: `do_something(data)`:



Предположим, что данные – это структура описывающая клиента банка. Если функция захочет поменять только ФИО, то она всё равно должна знать информацию о всей структуре, т.к. операции чтения и записи зависят от расположения полей в памяти. Любое изменение данных потребует перекомпиляции кода функций, которые эти данные используют, а их перекомпиляция потребует перекомпиляции кода всех, кто использует эти функции и т.д.



Объектная модель (слабая связность)

ООП-модель: программа – это сеть объектов, которые обмениваются сообщениями. Вычисление – это обмен сообщениями: `object.do_something()`.

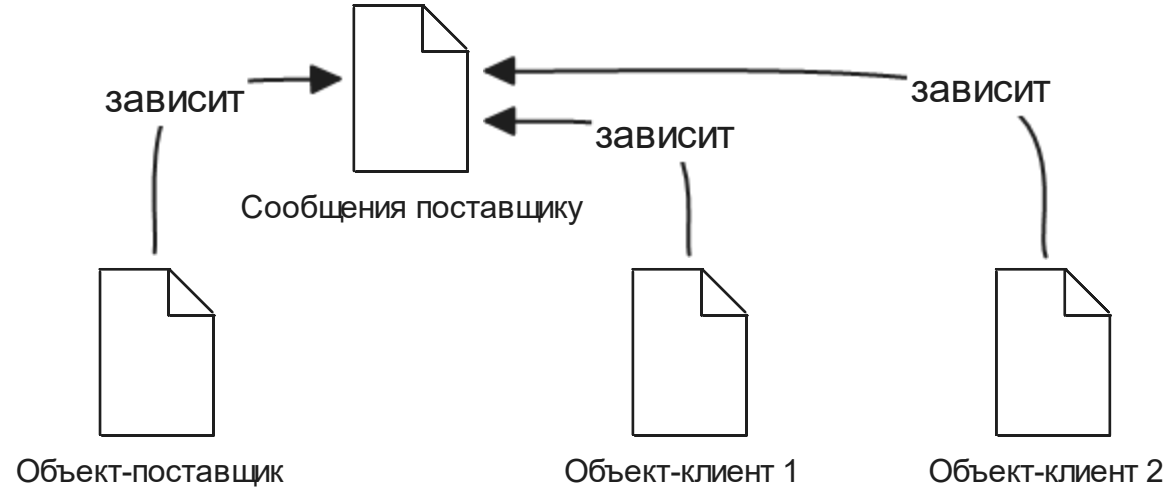
В идеале, объекты могут посылать сообщения в общую очередь и вообще не знать друг о друге ничего. Тот кто может отреагировать на сообщение увидит его в очереди и выполнит действие, а затем отправит ответное сообщение обратно в очередь:



Объектная модель (слабая связность)

Чаще используется другая модель. Мы раскрываем только минимально необходимую информацию об объекте-поставщике услуги – только список сообщений на которые он умеет реагировать и объект-клиент будет зависеть только от этого списка, но не от самого объекта-поставщика. Поэтому никакое изменение объекта-поставщика (не влияющие на список сообщений) не приведёт к необходимости перекомпиляции объекта-клиента.

Гарантируя неизменность списка сообщений, мы гарантируем независимость объектов друг от друга, а следовательно и возможность свободной их модификации:



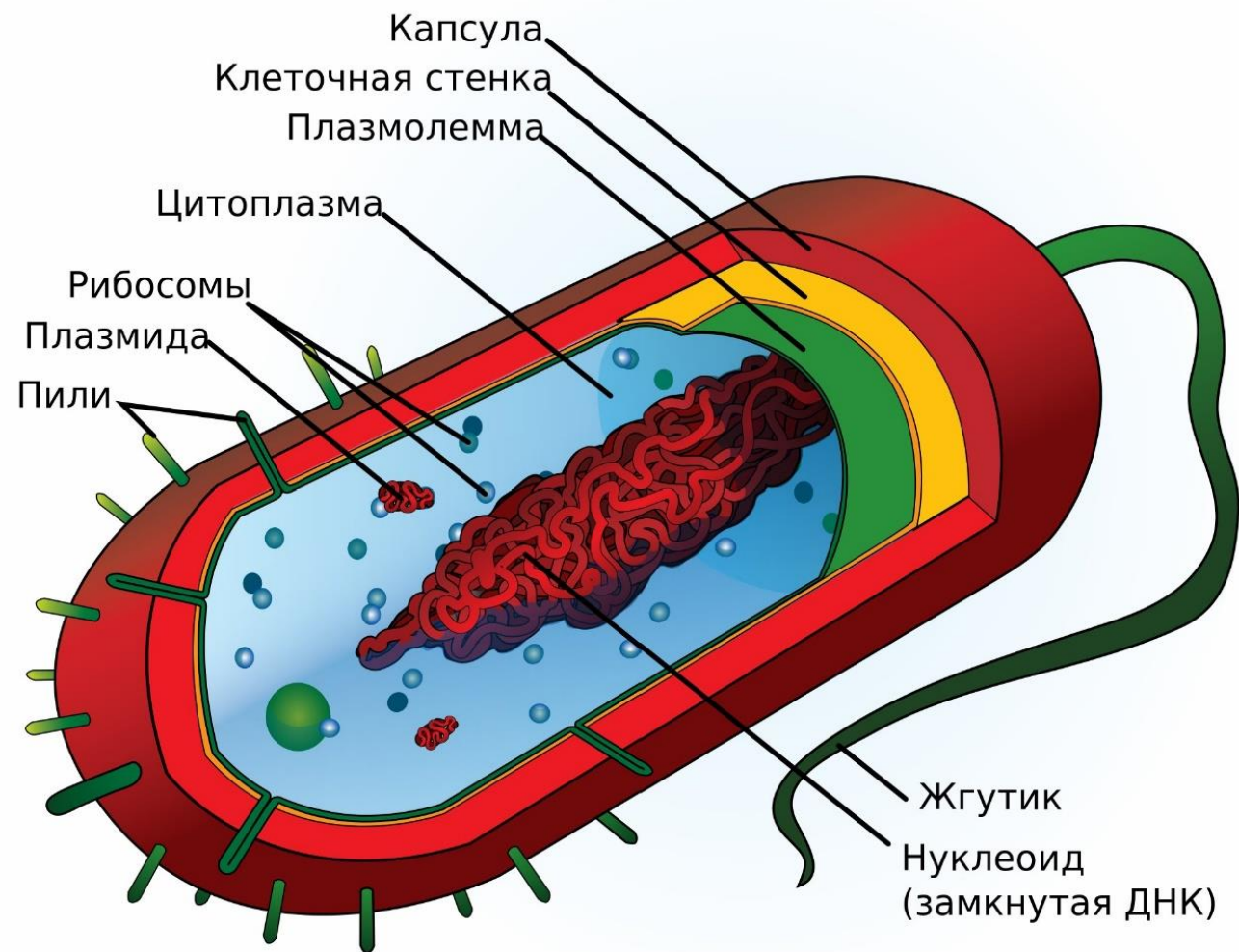
Инкапсуляция

Инкапсуляция

Объект – сущность отделённая от внешнего "мира" границей. У объекта есть своё состояние и он сам решает менять ли его или нет, реагировать на сообщение или нет. Состояние объекта и механизмы изменяющие его находятся внутри самого объекта – инкапсулированы в объекте.

Слово «инкапсуляция» происходит от латинского in (в) capsula (коробочка, оболочка) – процесс складывания в коробку; объединение в одну логическую единицу. В случае программирования эта логическая единица обычно называется класс.

Объект – био аналогия



Инкапсуляция

Инкапсуляция – это один из основных принципов ООП.

Принцип (основа, начало) – это идея которой люди руководствуются при выборе своего поведения. В нашем случае – философия проектирования класса и программной системы в целом.

Т.е. разрабатывая программы, вы должны держать в голове идею разделения кода на компоненты, которые как можно меньше зависят от внешнего мира и которые предпосылают внешнему миру только необходимый минимум информации о себе.

Состояние

Предположим есть простая структура:

```
struct Point{  
    int x;  
    int y;  
};
```

Любые комбинации значений x и y дают существующую точку в пространстве. Невозможно подобрать такие x и y , чтобы структура стала не валидной.

Теперь рассмотрим вторую структуру:

```
struct BankCustomer {  
    double account_balance; // Собственные средства на счете  
    double credit_limit;    // Кредитный лимит  
    vector<double> holds;   // Вектор замороженных сумм  
};
```

Если добавить в вектор `holds` значения сумма которых превышает `account_balance + credit_limit`, то структура перейдёт в недопустимое состояние. Аналогично, если `account_balance` сделать отрицательным и ещё много других комбинаций приведёт структуру в состояние изначально не предусмотренное разработчиками.

Инвариант

Т.е. экземпляры второй структуры подчиняются некоторому набору **правил**, которые НЕ должны нарушаться на протяжении всего жизненного цикла экземпляра. Такие правила обычно называют инвариантами класса/объекта.

Инвариант — свойство, остающееся неизменным при любых преобразованиях экземпляра. Например, полная энергия замкнутой системы – инвариант.

Жизненный цикл объекта — это последовательность этапов существования экземпляра от момента его создания до уничтожения. Жизненный цикл состоит из:

1. Создание (выделение памяти и инициализация);
2. Использование (работу с экземпляром);
3. Уничтожение (вызов деструктора и освобождение памяти).

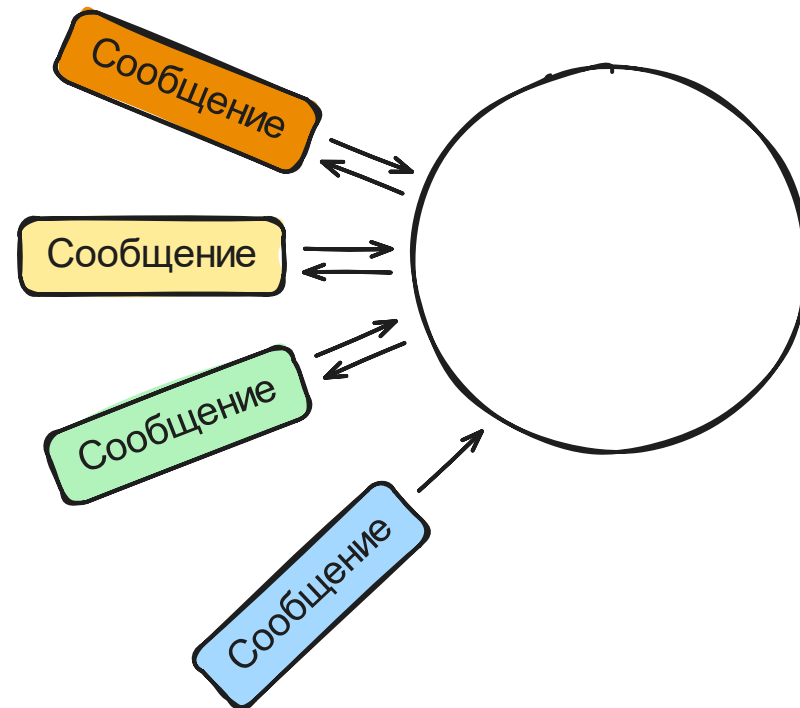
Т.е. в процессе создания, использования и уничтожения экземпляра структуры должен производиться контроль сохранения инвариантов.

Примеры инвариантов: `summa_elements holds <= account_balance + credit_limit; account_balance >= 0; credit_limit >= 0.`

Сообщения

В идеальной модели, внешняя система вообще ничего не знает про объект. На какие-то сообщения он реагирует, на какие-то нет. Это даёт возможность гибко менять объекты в процессе работы системы, добавляя/удаляя или изменяя его реакцию на сообщения, и тем самым гибко менять саму систему. Но на практике реализовать такое довольно "дорого".

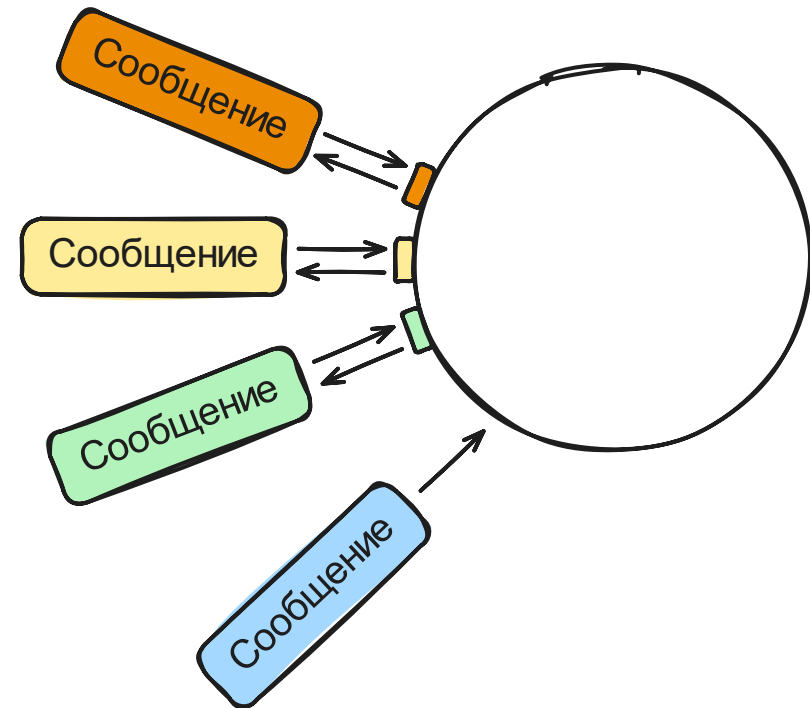
```
forrest.sendMessage("run");
```



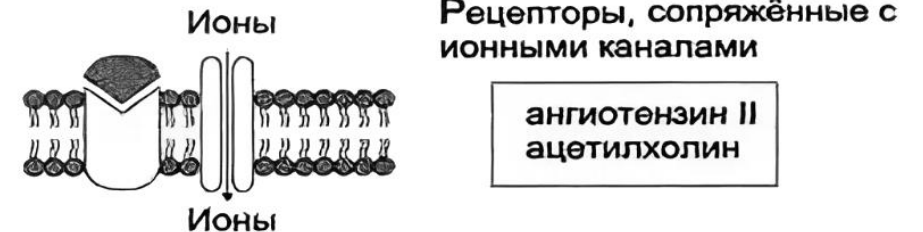
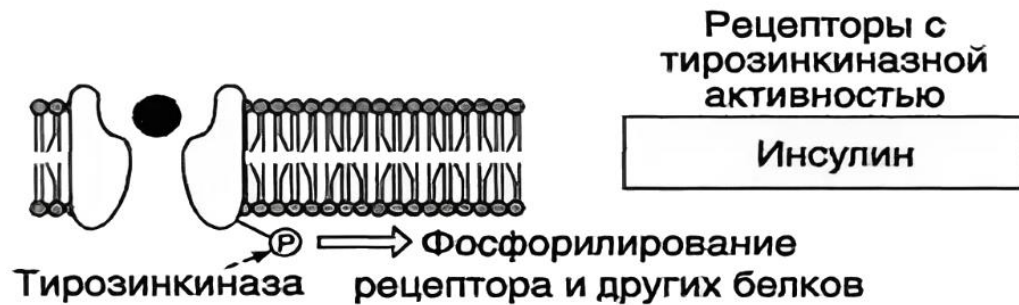
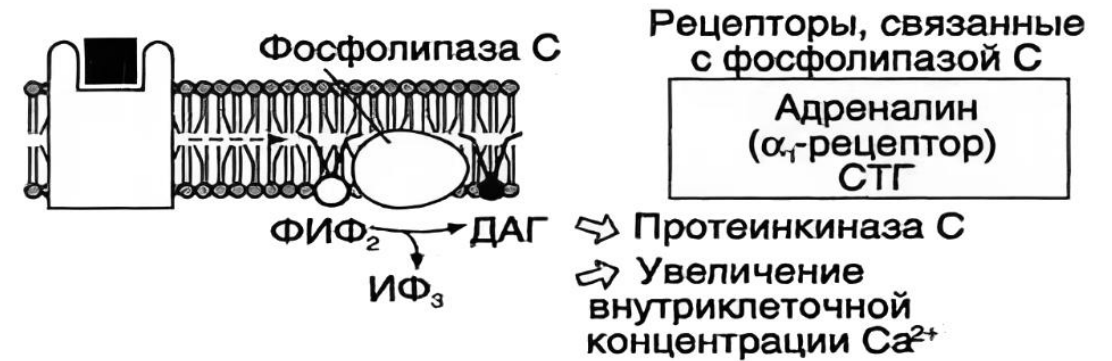
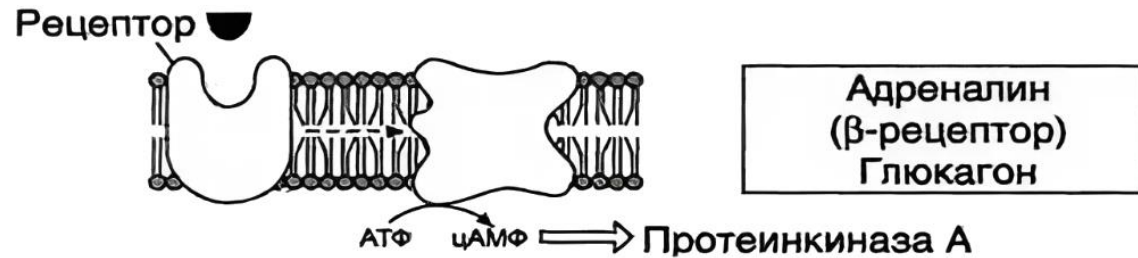
Сообщения - интерфейс

Как правило мы заранее определяем список сообщений на которые объект должен реагировать и этот список называют интерфейсом. Никакие другие сообщения объекту в принципе послать невозможно. Объектам-клиентам необходим этот список, чтобы понимать какие сообщения можно отправлять нашему объекту. Таким образом часть информации об объекте всё таки должна быть видна наружу:

```
class Human{  
    int speed;          // состояние  
public:  
    void run();         //  
    void stop();        // интерфейс  
    int getSpeed();     //  
} forrest;  
  
forrest.run();
```



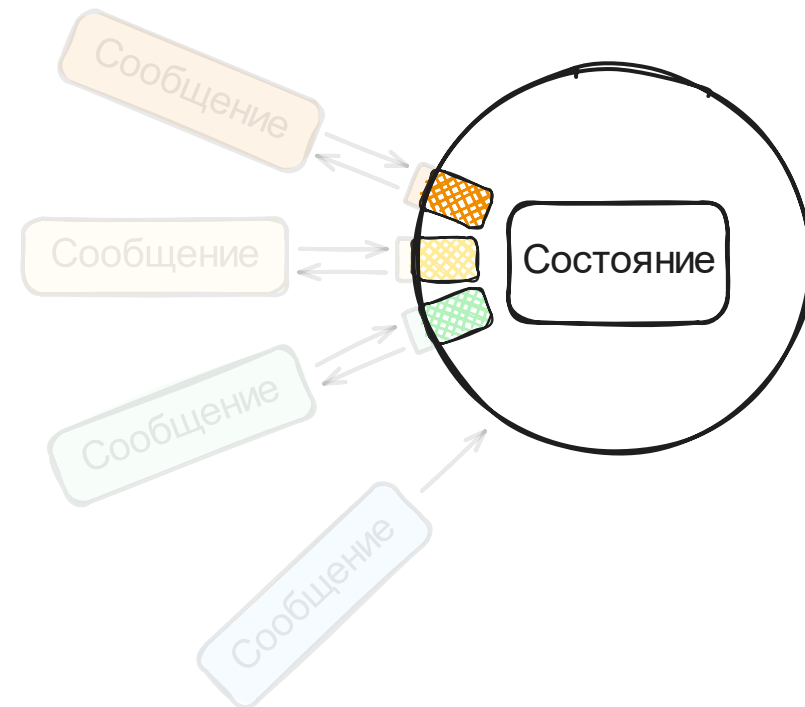
Интерфейс клетки (реагирует только на опред. сообщения)



Сообщения - реализация

Но вот то, как именно происходит реакция на сообщение объектам-клиентам знать не нужно, поэтому эта часть должна быть скрыта от них. Таким образом, хоть мы и не можем менять список сообщений на которые реагирует объект, мы можем менять саму реакцию.

```
void Human::run(){  
    speed = 42;  
}  
  
void Human::stop(){  
    speed = 0;  
}  
  
int Human::getSpeed(){  
    return speed;  
}
```



Контракт

Каждый метод взаимодействующий с внешней системой должен соблюдать набор правил. По аналогии с бизнесом этот набор правил называют Контрактом.

Контракт может включать в себя:

- Предусловия (requirements) – набор требований, которые мы предъявляем к внешней системе и к входным данным. Например: количество обращений в минуту, формат данных, размер, мин/мах значения и т.д. Если требования не исполнены, то метод не исполняет свои обязательства (тело). Это наши требования мы должны их проверять;
- Постусловия (guarantees) – набор гарантий, на которые внешняя система может рассчитывать вызывая метод. Например: формат результата, мин/мах значения результата, время работы метода и потребление памяти, исключения бросаемые методом и т.д. Это наши гарантии, мы должны их соблюдать;
- Сохранение инвариантов класса. Чтобы там ни было мы должны сохранять инварианты и проектировать контракт предусматривая эту необходимость.

Контракт

Формулировать контракт лучше всего на естественном языке, т.к. таким образом вы однозначно описываете свои намерения и используете язык бизнеса понятный и программистам и специалистам предметникам.

Контракт должен быть сформулирован как можно конкретнее: «Сумма средств на счету клиента **может** быть больше нуля» - А может и не быть? А какие условия? и т.д.

Требование: `amount > 0`. Название переменной или метода может быть выбрано неудачно, быть изменено в последствии, или просто отсутствовать для уменьшения дублирования кода. Текст: «Сумма средств на счету клиента должна быть больше нуля» чётко и однозначно даёт понять о предъявляемом требовании.

Инкапсуляция в C++

Обычно под инкапсуляцией понимают:

- Объединение данных (состояние) и методов для работы с этими данными (реакция на сообщения, поведение) в одной логической единице. В C++ это класс `class` и структура `struct`. Состояние описывается полями класса, а поведение – методами;
- Ограничения доступа к членам класса со стороны внешних компонентов. Иногда говорят просто сокрытие данных. В C++ за это отвечают модификаторы доступа: `public`, `protected`, `private` и некоторые другие механизмы.

Инкапсуляция в C++

В C++ принято для класса создавать пару файлов:

- `.h` – для размещения интерфейсной части класса. Затем этот файл подключается к клиентскому коду;
- `.cpp` – для размещения реализации методов класса. Этот файл может быть в виде исходников или скомпилирован в объектный файл или библиотеку.

Такое разделение позволяет свободно изменять реализацию класса без необходимости изменять клиентский код. Если реализация собрана в динамическую библиотеку, то её достаточно заменить вообще без какого-либо изменения в клиентском приложении и без перекомпиляции программы. Если реализация в виде статической библиотеки, то достаточно выполнить фазу линковки, без изменения кода клиентского приложения.

В качестве "бонуса", можно не предоставлять исходный код реализации, а только скомпилированные бинарный файл.

Для шаблонов классов так сделать не получится. Реализацию шаблона класса придётся размещать в `.h` файле, т.к. компилятору она нужна для инстанцирования. Как обходной вариант можно инстанцировать класс вручную и тогда его реализацию можно перенести в `.cpp` файл.

Инкапсуляция в C++

```
human.cpp

#include "human.h"

void Human::run(){
    speed = 42;
}

void Human::stop(){
    speed = 0;
}

int Human::getSpeed(){
    return speed;
}
```

Реализация
Можно менять



```
human.h

class Human{
    int speed;      // состояние
public:
    void run();     // 
    void stop();    // интерфейс
    int getSpeed(); // 
};
```

```
client.cpp

#include "human.h"

int main(){
    Human h1, h2;
    h1.run();
    h2.run();
}
```

Клиентский код
Не изменяется



Идиома Pimpl (Pointer to Implementation) в C++

Как видно, в предыдущем коде, в файле "human.h", кроме интерфейса присутствует часть состояния объекта, что не очень хорошо с точки зрения принципа инкапсуляции. Т.к. при изменении переменных описывающих состояние объекта, придётся пересобрать пользовательский код. И кроме того, пользователь может "привязать" свой код к наличию именно этих переменных.

Классическим решением этой проблемы в C++ является идиома *Pointer to Implementation* (указатель на реализацию). Она обеспечивает очень хорошую инкапсуляцию.

Идея заключается в том, что основной класс мы называем реализацией (*Impl*) и дополнительно создаём обёртку которая служит как интерфейс (обработчик сообщений от других объектов). Как правило обёртка под капотом просто вызывает соответствующие методы реализации и хранит указатель на реализацию. Нужен именно указатель, т.к. компилятору не нужно знать ничего о самом классе и поэтому мы можем полностью вынести его в `.cpp` файл или библиотеку.

Проблемы данного подхода: дополнительное разыменование указателя при каждом вызове; маленький шанс встраивания методов.

Инкапсуляция в C++

```
human.cpp

#include "human.h"

struct Human::Impl {
    int speed;

    void run()    { speed = 42;  }
    void stop()   { speed = 0;   }
    int getSpeed(){ return speed; }
};

Human::Human(): pimpl(new Impl()){
}

Human::~Human(){
    delete pimpl;
}

void Human::run(){
    pimpl->run();
}

void Human::stop(){
    pimpl->stop();
}

int Human::getSpeed(){
    return pimpl->getSpeed();
}
```

```
human.h

class Human{
    class Impl; // неполный тип. Просто объявление
    Impl *pimpl; // поле-указатель
public:
    Human();
    ~Human();

    void run();
    void stop();
    int getSpeed();
};
```

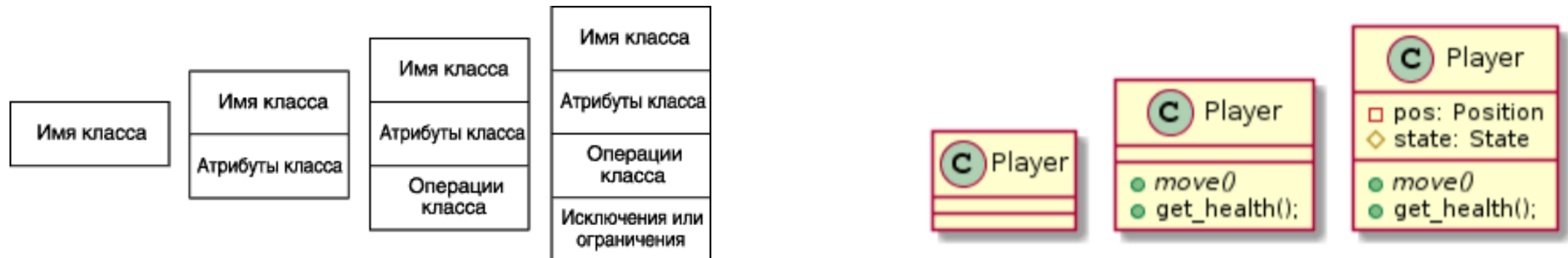
```
client.cpp

#include "human.h"

int main(){
    Human h1, h2;
    h1.run();
    h2.run();
}
```

Нотация UML

В нотации *UML* класс изображается в виде прямоугольника, который дополнительно может быть разделен горизонтальными линиями на секции. В этих секциях могут указываться имя класса, атрибуты и операции класса.



В зависимости от параметра видимости элемент класса может быть:

- приватным (private, доступен только внутри класса) — задается символом «минус» (-), может отображаться в виде квадрата;
- защищенным (protected, доступен внутри класса, а также внутри классов-наследников) — задается символом «решетка» (#), может отображаться в виде ромба;
- открытым (public, доступен всем) — задается символом «плюс» (+), может отображаться в виде круга.

Виртуальный метод и имя абстрактного класса выделяются курсивом, а статический метод — подчеркивается.

Отношения между классами в UML

Тип отношения	UML-синтаксис		Краткая семантика
	источник	цель	
Зависимость			Исходный элемент зависит от целевого элемента и изменение последнего может повлиять на первый.
Ассоциация			Описание набора связей между объектами.
Агрегация			Целевой элемент является частью исходного элемента.
Композиция			Строгая (более ограниченная) форма агрегирования.
Включение			Исходный элемент содержит целевой элемент.
Обобщение (наследование)			Исходный элемент является специализацией более обобщенного целевого элемента и может замещать его.
Реализация			Исходный элемент гарантированно выполняет контракт, определенный целевым элементом.

Наследование

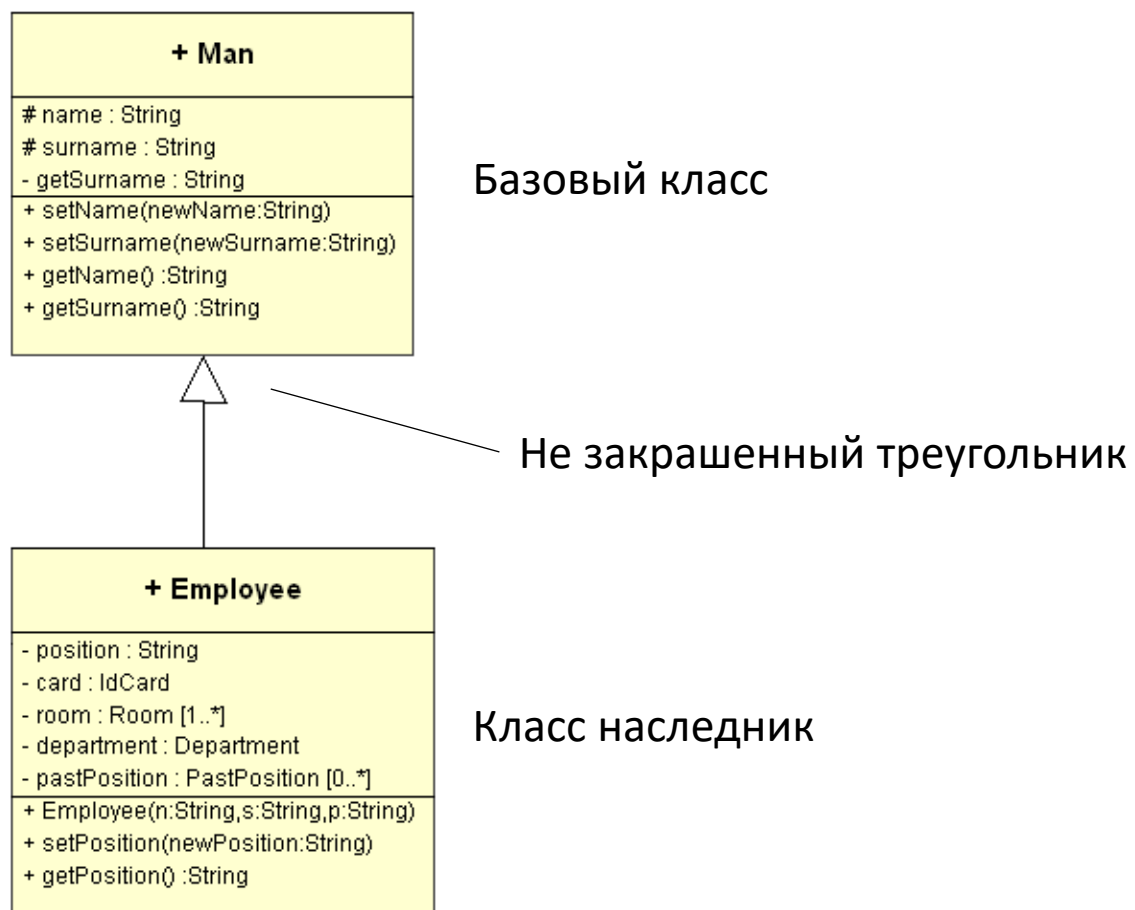
Наследование

Наследование – это механизм создания новых объектов или классов на основе существующих объектов (прототипное наследование) или классов (наследование на основе классов).

Существующий класс называют: родительским классом, базовым классом, классом предком, супер-классом (super с лат. над, выше), надкласс;

Новый класс называют: дочерним, производным классом, классом наследником, подклассом.

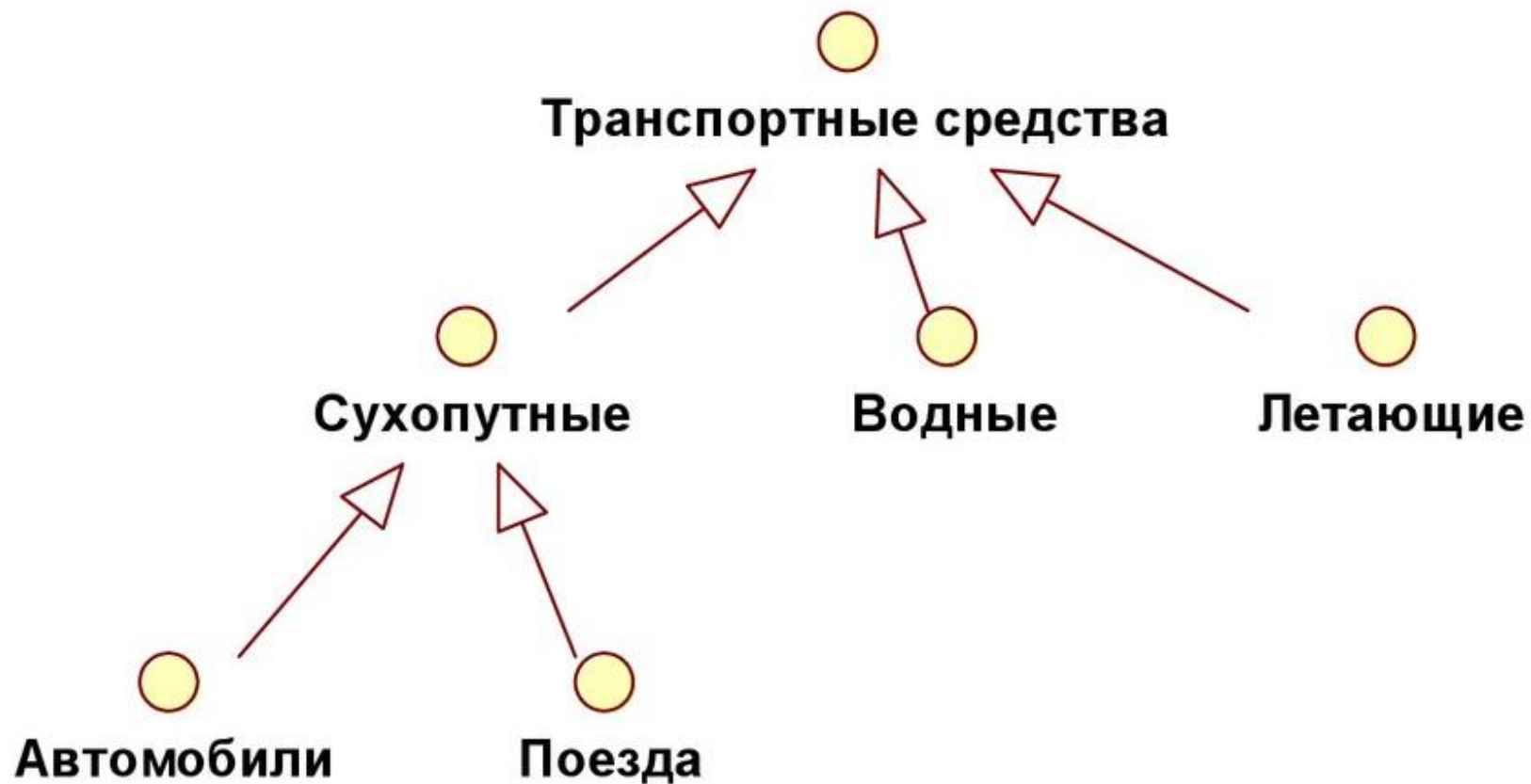
Наследование (обобщение) в UML



Примеры наследования



Примеры наследования



Наследование

Наследование можно разделить на:

- Прототипное наследование (prototype-based inheritance);
- Наследование на основе классов (class-based inheritance).

Еще можно выделить:

- Одиночное наследование. Разрешено, чтобы у класса был только один предок;
- Множественное наследование. Разрешено, чтобы у класса было больше одного предка. При этом свойства и методы наследуются у всех.

Принцип наследования – это только про создание одних классов на основе других с возможностью получения полей и методов предков с целью уменьшения дублирования кода и больше ни про что. Никаких правил и ограничений что от чего наследовать нет.

Существует так же Подтипизация (Subtyping) – процесс построения иерархии связанных типов, каждый из которых является подтипом предыдущего. Вот здесь уже существует набор правил.

Многие языки (в том числе и C++) автоматически связывают классы в иерархию типов при наследовании и нет возможности от этого отказаться. Но важно понимать, что наследование и подтипизация – это разные процессы и их можно использовать независимо друг от друга.

Прототипное наследование

При прототипном наследовании отсутствует понятие класса, есть только объекты. У объекта может быть объект-прототип или может не быть. Если объект-прототип есть, то при обращении к свойству или методу текущего объекта выполняется поиск сначала в текущем объекте, а затем в объекте прототипе, а затем в прототипе прототипа и т.д. Объект-прототип может быть изменён в процессе работы программы.

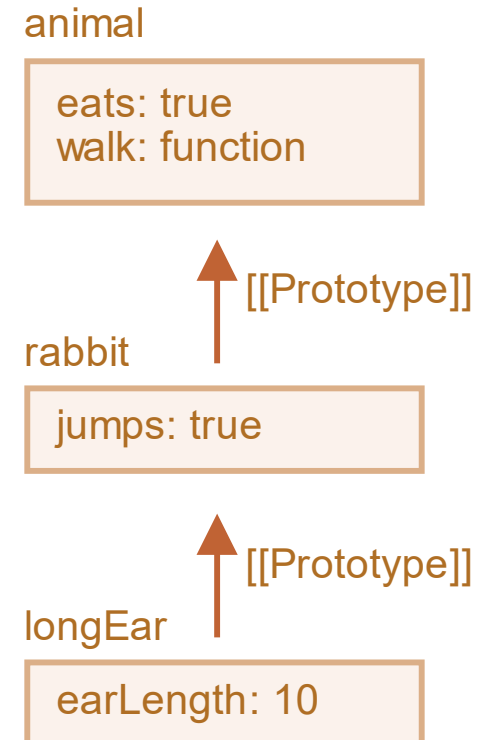
```
let animal = {
  eats: true,
  walk() {
    alert("Animal walk");
  }
};

let rabbit = {
  jumps: true,
  __proto__: animal
};

let longEar = {
  earLength: 10,
  __proto__: rabbit
};

// walk взят из цепочки прототипов
longEar.walk(); // Animal walk
alert(longEar.jumps); // true (из rabbit)
```

JS



Наследование на основе классов

При наследовании на основе классов существуют классы (определяют структуру данных и поведение) и объекты (является носителем данных). Методы и структура полей наследуются через цепочку классов. Объект содержит данные описанные в его классе и во всех предках.

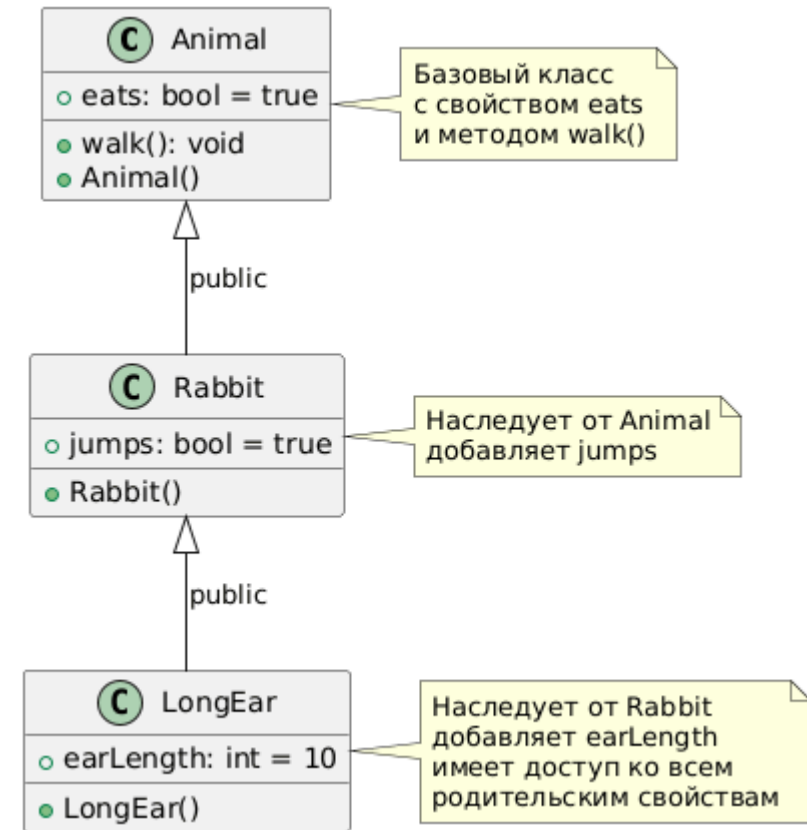
```
class Animal {
public:
    bool eats = true;

    void walk() {
        cout << "Animal walk" << endl;
    }
};

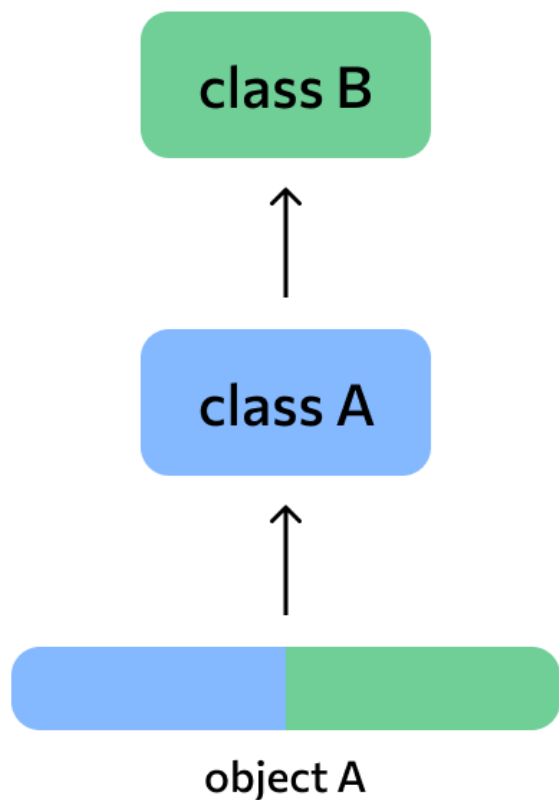
class Rabbit : public Animal { // Наследование от Animal
public:
    bool jumps = true;
};

class LongEar : public Rabbit { // Наследование от Rabbit
public:
    int earLength = 10;
};

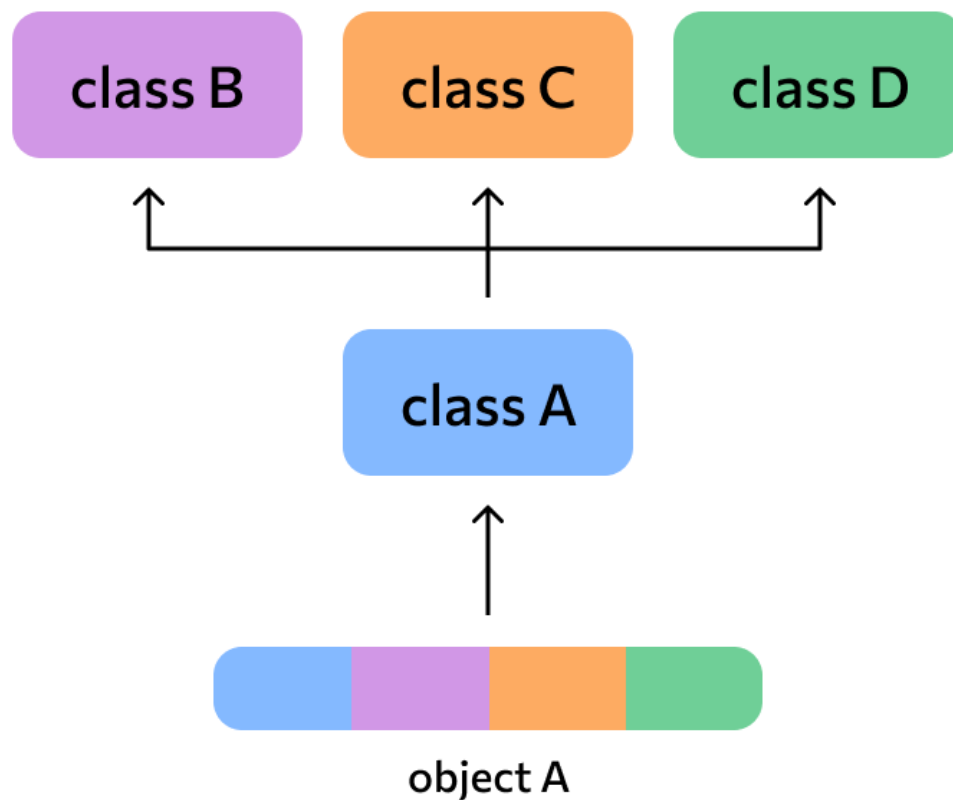
LongEar longEar;
longEar.walk(); // Animal walk (унаследовано от Animal)
longEar.jumps; // 1 (true, унаследовано от Rabbit)
```



Виды наследования в C++



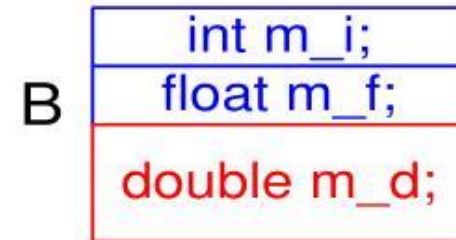
Одиночное
наследование



Множественное
наследование

Одиночное наследование в C++

```
class A{  
    int    m_i;  
    float m_f;  
};  
  
class B: A{  
    double m_d;  
};  
  
class C: B{  
    long m_l;  
};
```

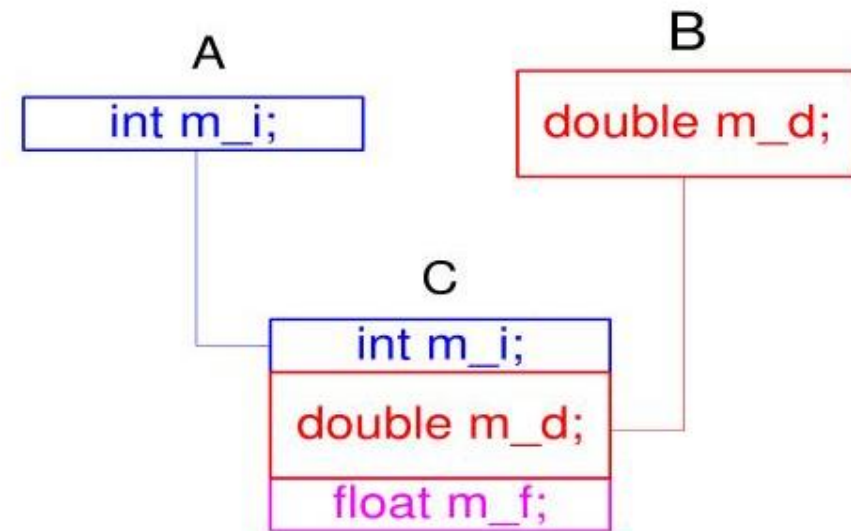


// Класс B получает все поля и методы класса A;

// Класс C получает все поля и методы класса B (включая всё из класса A).

Множественное наследование в C++

```
class A{  
    int m_i;  
};  
  
class B {  
    double m_d;  
};  
  
class C: A, B{  
    float m_f;  
};
```



// Класс C получает все поля и методы класса B и класса A.

Тип наследования

```
class Der: public Base{};    // Открытое наследование  
class Der: protected Base{}; // Защищённое наследование  
class Der: private Base{};   // Закрытое наследование
```

// Независимо от того Base class или struct

```
class Der: /* private */ Base{};  
struct Der: /* public */ Base{};
```

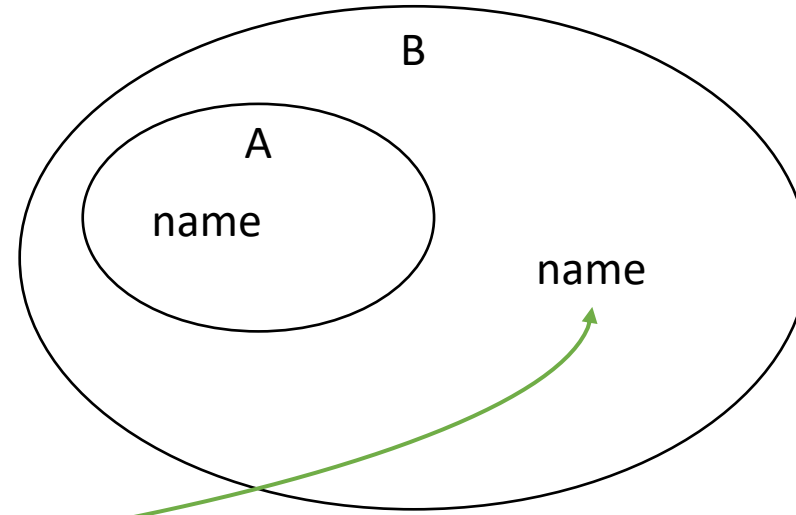
// Как правило используется public наследование

Модификатор доступа в классе наследнике

		Модификатор члена базового класса		
		public	protected	private
Тип наследования	public	public	protected	нет доступа
	protected	protected	protected	нет доступа
	private	private	private	нет доступа

Правила поиска имён

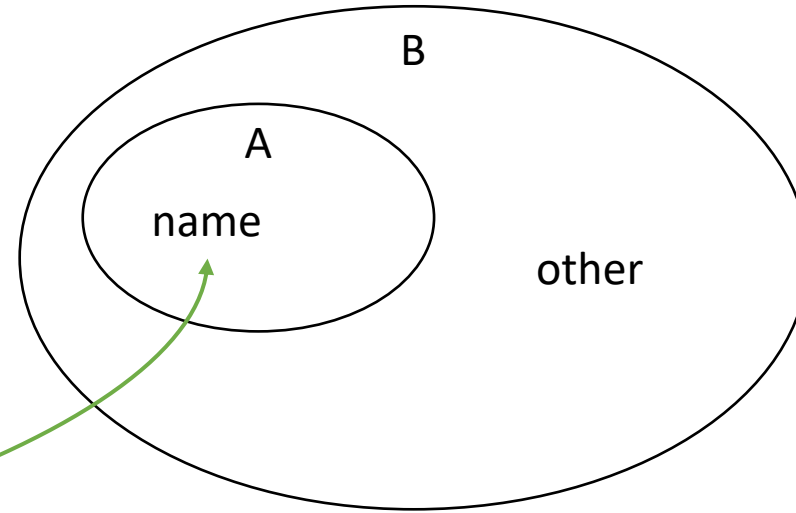
```
class A{  
public:  
    int name = 10;  
};  
  
class B: public A{  
public:  
    int name = -10;  
};  
  
B object;  
object.name = 5;
```



Поиск имени выполняется начиная с класса B. Т.к. имя name в нём есть, то дальнейший поиск не происходит и `object.name = 5;` присвоит 5 собственной переменной name класса B.

Правила поиска имён

```
class A{  
public:  
    int name = 10;  
};  
  
class B: public A{  
public:  
    int other = -10;  
};  
  
B object;  
object.name = 5;
```



Поиск имени выполняется начиная с класса B. Т.к. имени `name` в нём нет, то поиск продолжается в родительских классах, т.е. в классе A. Поэтому `object.name = 5;` присвоит 5 переменной `name` доставшейся классу B от класса A.

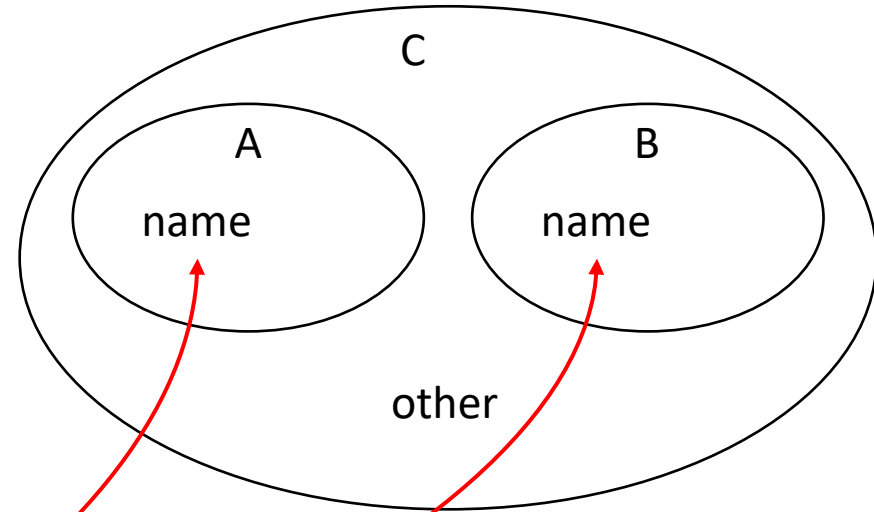
Правила поиска имён

```
class A{  
public:  
    int name = 10;  
};
```

```
class B{  
public:  
    int name = -10;  
};
```

```
class C: public A, public B{  
public:  
    int other;  
};
```

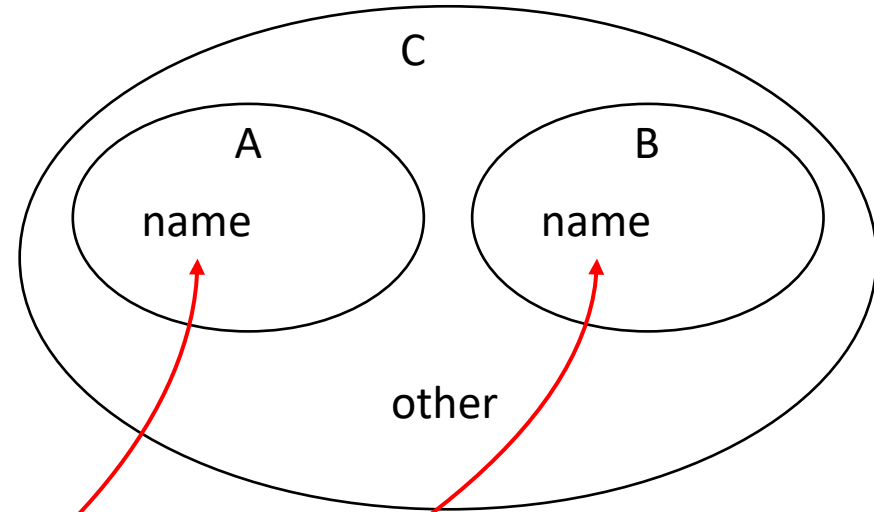
```
C object;  
object.name = 5;
```



Поиск имени выполняется начиная с класса C. Т.к. имени name в нём нет, то поиск продолжается в родительских классах, т.е. в классе A и классе B. Имя name присутствует в обоих классах, поэтому компилятор не может выбрать какое-то одно. Т.е. такой код приведёт к ошибке. Если у класса A не будет имени name, но будет у его предка, то результат такой же.

Правила поиска имён

```
class A{  
private:  
    int name = 10;  
};  
  
class B{  
public:  
    int name = -10;  
};  
  
class C: public A, public B{  
public:  
    int other;  
};  
  
C object;  
object.name = 5;
```



На процесс поиска имён квалификаторы доступа не влияют. Несмотря на то, что у класса **A** имя **name** находится в приватной секции и 100% не может быть доступно в точке вызова, а у класса **B** есть доступное имя **name**, мы получаем ошибку, по той же причине, что и на предыдущем слайде.

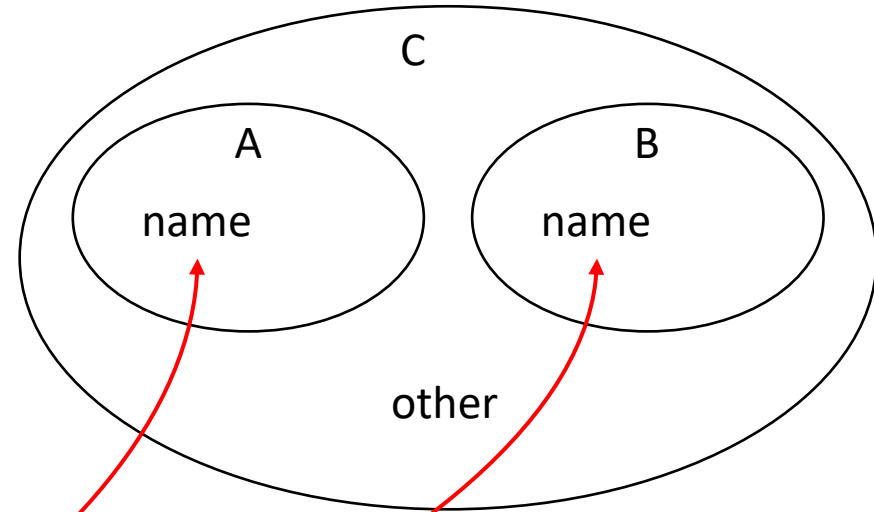
Правила поиска имён

```
class A{  
public:  
    int name = 10;  
};
```

```
class B{  
public:  
    void name(){}  
};
```

```
class C: public A, public B{  
public:  
    int other;  
};
```

```
C object;  
object.name = 5;
```



На процесс поиска имён не влияет метод мы ищем или поле, ищется просто имя. Несмотря на то, что у класса **A** имя **name** это поле и операция присваивания возможна, а у класса **B** **name** — это метод и присваивание не возможно, компилятор не может выбрать и сообщит об неопределённости.

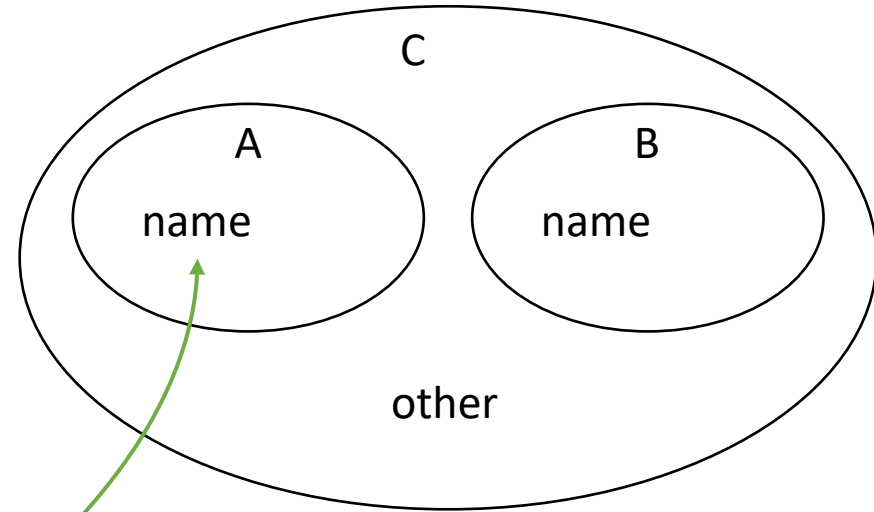
Правила поиска имён

```
class A{  
public:  
    int name = 10;  
};
```

```
class B{  
public:  
    int name = -10;  
};
```

```
class C: public A, public B{  
public:  
    int other;  
};
```

```
C object;  
object.A::name = 5;
```



Квалификатор `A::` однозначно определяет какое имя `name` нужно использовать. Тут всё ОК

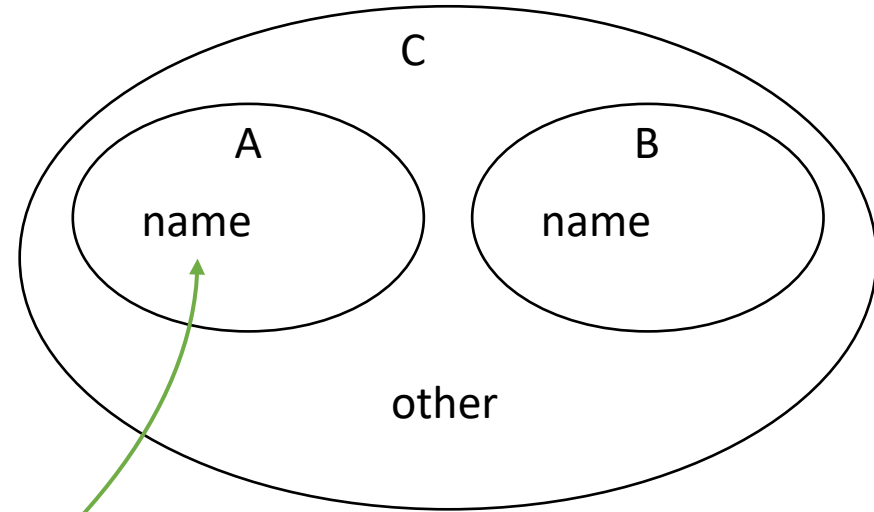
Правила поиска имён

```
class A{  
public:  
    int name = 10;  
};
```

```
class B{  
public:  
    int name = -10;  
};
```

```
class C: public A, public B{  
public:  
    int other;  
};
```

```
C object;  
object.A::name = 5;
```



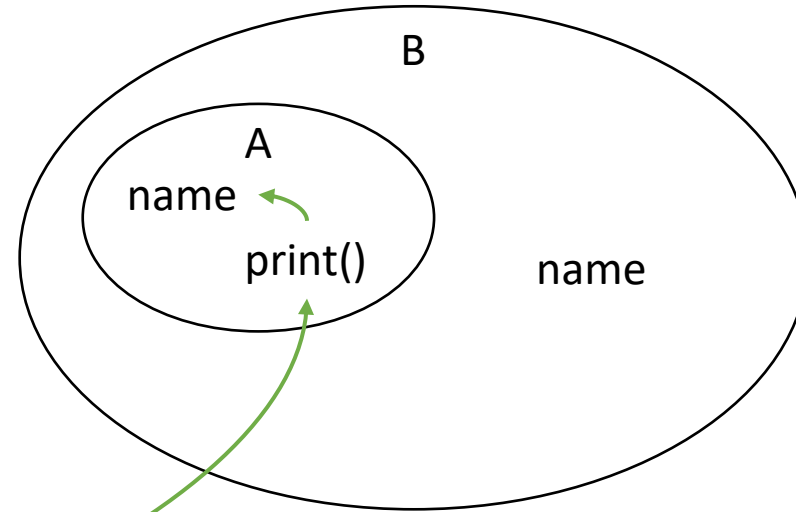
Квалификатор `A::` однозначно определяет какое имя `name` нужно использовать. Тут всё ОК

Правила поиска имён

```
class A{  
public:  
    int name = 10;  
    void print(){  
        cout << name << endl;  
    }  
};
```

```
class B: public A{  
public:  
    int name = -10;  
};
```

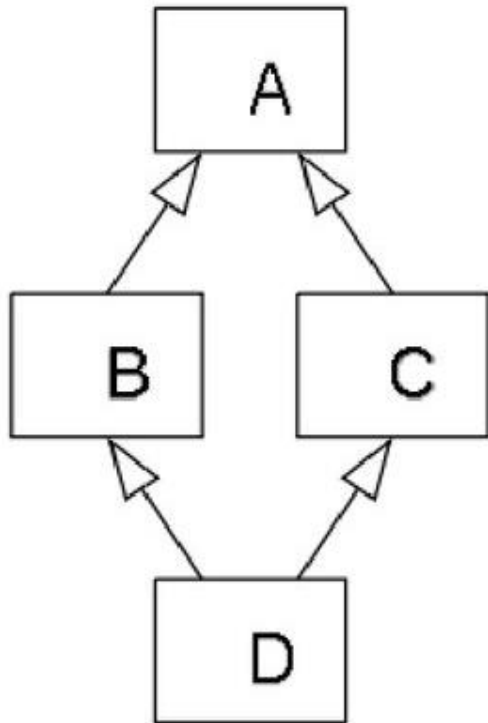
```
B object;  
object.print();
```



Метод print из класса A будет начинать поиск имён со своего класса и далее в его предках. О поле name класса наследника, как и о наследнике в целом, он ничего не знает.

Ромбовидное наследование

Проблема ромба
("diamond problem")



- Члены-данные класса A в объектах класса D в двух экземплярах
- Обращение к членам класса A в классе D невозможно без уточнения непосредственного наследника (B или C)
- Порядок наследования может привести к изменению семантики
- Решение
`class B : virtual public A`

Ромбовидное наследование

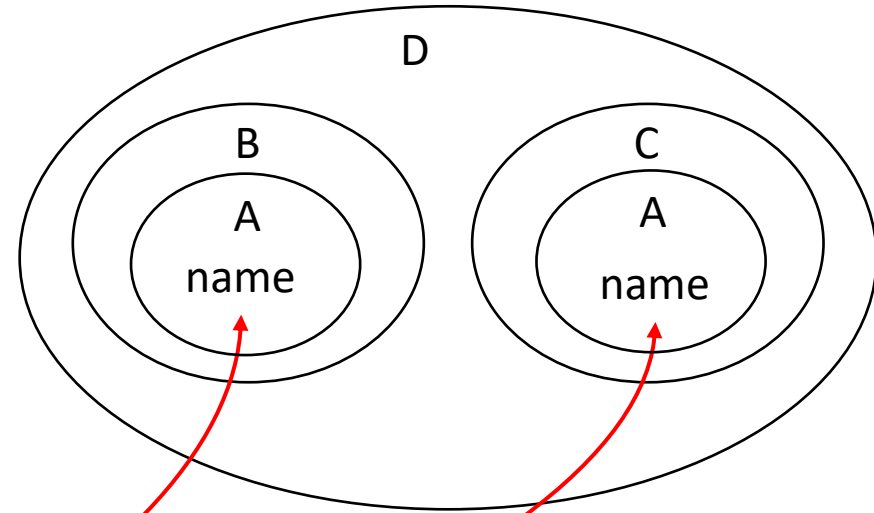
```
class A{  
public:  
    int name = 10;  
};
```

```
class B: public A{  
};
```

```
class C: public A{  
};
```

```
class D: public B, public C{  
};
```

```
D object;  
object.name = 5;
```



Процесс поиска имён начинается с класса **D** затем продолжается в классах **B** и **C**. Т.к. и **B** и **C** внутри себя содержат класс **A** то имя **name** будет найдено и там и там и мы получим ошибку.

Виртуальное наследование

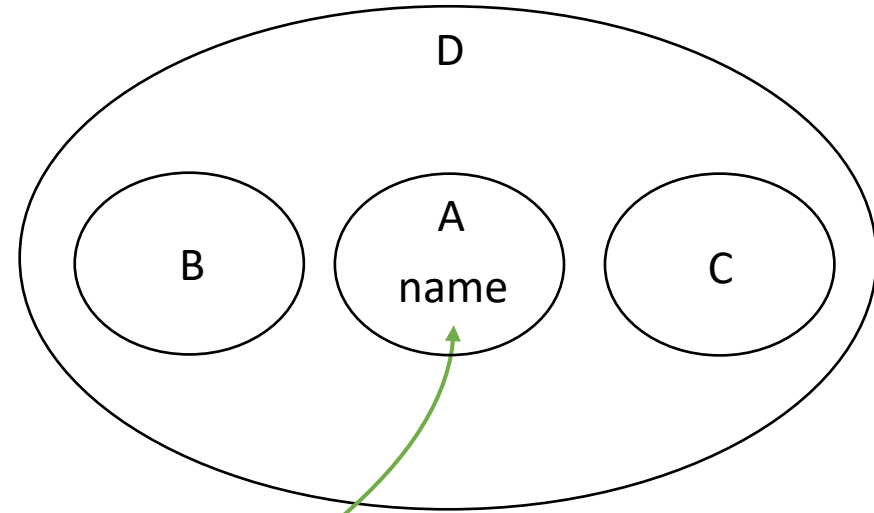
```
class A{  
public:  
    int name = 10;  
};
```

```
class B: public virtual A{  
};
```

```
class C: public virtual A{  
};
```

```
class D: public B, public C{  
};
```

```
D object;  
object.name = 5;
```



В случае виртуального наследования класс **A**, который наследуется виртуально, будет присутствовать в наследнике в единственном числе в виде отдельного класса (как бы третий предок)

Переопределение

```
class A{
public:
    int name = 10;
    void print(){
        cout << "A" << endl;
    }
};
```

```
class B: public A{
public:
    int name = -10;
    void print(){
        cout << "B" << endl;
    }
};
```

```
B object;
cout << object.name; // -10
object.print();      // B
```

Таким образом правила поиска имён приводят к тому, что наличие метода или поля в классе наследнике **переопределяет** одноимённый метод предка.

В случае наличия в классе предке нескольких перегрузок метода, метод в классе наследнике **переопределяет их все**. Т.е. они не будут доступны, а будет доступен только метод наследника.

Иногда вместо термина "переопределяет", для НЕ виртуальных методов говорят – "перекрывает", чтобы подчеркнуть, что метод не виртуальный. Разница в том, что в случае каста указателя/ссылки на наследник к предку, виртуальный метод будет показывать тоже, что и раньше, а не виртуальный, покажет вывод предка (вроде как он только скрывал метод предка, а не заменял его).

В целом и то и то является переопределением.

Ключевое слово `final` при объявлении класса

```
class A{  
};
```

```
class B final: public A{  
};
```

```
class C: public B{ // Ошибка  
};
```

Если объявить класс с ключевым словом `final`, то наследование от данного класса будет запрещено.

Преобразование объектов в цепочке наследования

```
#include <iostream>
#include <string>

class A{
public:
    void print(){
        std::cout << "A" << std::endl;
    }
};

class B: public A{
public:
    void print(){
        std::cout << "B" << std::endl;
    }
};

int main(){
    B obj_b;
    static_cast<A>(obj_b).print();    // A
    static_cast<A&>(obj_b).print();   // A
    static_cast<A*>(&obj_b)->print(); // A

    A obj_a;
    //static_cast<B>(obj_a).print(); // Ошибка. Преобразование не определено
    static_cast<B&>(obj_a).print();   // Потенциальная ошибка
    static_cast<B*>(&obj_a)->print(); // Потенциальная ошибка
}
```

Преобразование объектов в цепочке наследования вверх по иерархии от наследника к любому предку безопасно, хотя и не всегда имеет смысл. Его называют UpCast.

Преобразование объектов в цепочке наследования вниз по иерархии от предка к наследнику как правило опасно. Его называют DownCast.

DownCast можно считать безопасным, если объект не содержит полей, а только методы или объект изначально был целевым. Т.е. был объект, выполнили UpCast и затем обратно DownCast.

Обычно выполняют преобразования не самих объектов, а указателей/ссылок.

`static_cast` применяется к экземплярам классов без виртуальных методов. Если есть виртуальные методы, нужно использовать `dynamic_cast`.

Преобразование объектов в цепочке наследования

```
#include <iostream>
#include <string>

class A{
public:
    void print(){
        std::cout << "A" << std::endl;
    }
};

class B: private A{ // или protected A
public:
    void print(){
        std::cout << "B" << std::endl;
    }
};

int main(){
    B obj_b;
    static_cast<A>(obj_b).print();    // Невозможно
    static_cast<A&>(obj_b).print();   // Невозможно
    static_cast<A*>(&obj_b)->print(); // Невозможно

    A obj_a;
    //static_cast<B>(obj_a).print();   // Невозможно
    static_cast<B&>(obj_a).print();    // Невозможно
    static_cast<B*>(&obj_a)->print();  // Невозможно
}
```

В случае приватного или защищённого наследования преобразования вверх или вниз по иерархии невозможны. Можно заставить компилятор это сделать при помощи `reinterpret_cast`, но не стоит этим злоупотреблять, т.к. `reinterpret_cast` не производит никаких проверок на допустимость преобразования и просто заставляет компилятор смотреть на тип как на другой.

Работает только с указателями и ссылками, но не со значениями.

Специальные методы

Конструктор

Конструктор по умолчанию

```
class A{  
    // Присутствует реализация конструктора от компилятора  
};  
  
class B: A{  
    // Присутствует реализация конструктора от компилятора  
};  
  
int main(){  
    B b;  
}
```

Конструктор класса наследника вызывает конструктор предка **перед** тем, как отработать.

Если конструктор объявленный пользователем отсутствует, то вызывается конструктор по умолчанию предоставленный компилятором.

Конструктор по умолчанию

```
#include <iostream>
#include <string>

class A{
    A(){
        std::cout << "A" << std::endl;
    }
};

class B: A{
    // Присутствует реализация конструктора от компилятора
};

int main(){
    B b; // Ошибка. Конструктор класса A приватный
}
```

Конструктор по умолчанию

```
#include <iostream>
#include <string>

class A{
public:
    A(){
        std::cout << "A" << std::endl;
    }
};

class B: A{
    // Присутствует реализация конструктора от компилятора. Ничего не выводит
};

int main(){
    B b; // Вывод: A
}
```

Конструктор по умолчанию

```
#include <iostream>
#include <string>

class A{
public:
    A(){
        std::cout << "A ";
    }
};

class B: A{
public:
    B(){ // Вызывается конструктор по умолчанию предка
        std::cout << "B" << std::endl;
    }
    B(int b){ // Вызывается конструктор по умолчанию предка
        std::cout << "B(" << b << ")" << std::endl;
    }
};

int main(){
    B b1; // Вывод: A B
    B b2(10); // Вывод: A B(10)
}
```

При объявлении своего (любого) конструктора в классе наследнике, перед ним автоматически вызывается конструктор по умолчанию класса предка.

Конструктор с параметром

```
#include <iostream>
#include <string>

class A{
public:
    A(int a){
        std::cout << "A(" << a << ")";
    }
};

class B: A{
public:
    B(){
        std::cout << "B" << std::endl;
    }
    B(int b){
        std::cout << "B(" << b << ")" << std::endl;
    }
};

int main(){
    B b1;      // Ошибка. Отсутствует конструктор по умолчанию
    B b2(10); // Ошибка. Отсутствует конструктор по умолчанию
}
```

В случае отсутствия конструктора по умолчанию в классе предке, другой конструктор наследником не вызывается, даже если он есть.

Конструктор явный вызов

```
#include <iostream>
#include <string>

class A{
public:
    A(int a){
        std::cout << "A(" << a << ") ";
    }
};

class B: A{
public:
    B():A(0){
        std::cout << "B" << std::endl;
    }
    B(int b):A(1){
        std::cout << "B(" << b << ")" << std::endl;
    }
};

int main(){
    B b1;        // A(0) B
    B b2(10);    // A(1) B(10)
}
```

В конструкторе наследника можно/нужно вызвать любой конструктор предка указав его явно в списке инициализаторов.

Конструктор в цепочке наследования

```
#include <iostream>
#include <string>

class A{
public:
    A(){
        std::cout << "A ";
    }
};

class B: A{
public:
    B(){
        std::cout << "B ";
    }
};

class C: B{
public:
    C(){
        std::cout << "C";
    }
};

int main(){
    C c;    // A B C
}
```

В случае цепочки предков они отработывают от самого первого предка последовательно к наследнику.

Конструктор порядок вызова

```
#include <iostream>
#include <string>

class A{
public:
    A(){
        std::cout << "A ";
    }
};

class B{
public:
    B(){
        std::cout << "B ";
    }
};

class C: A, B{
public:
    C(){
        std::cout << "C";
    }
};

int main(){
    C c;    // A B C <----
}
```

```
#include <iostream>
#include <string>

class A{
public:
    A(){
        std::cout << "A ";
    }
};

class B{
public:
    B(){
        std::cout << "B ";
    }
};

class C: B, A{
public:
    C(){
        std::cout << "C";
    }
};

int main(){
    C c;    // B A C <----
}
```

```
#include <iostream>
#include <string>

class A{
public:
    A(){
        std::cout << "A ";
    }
};

class B{
public:
    B(){
        std::cout << "B ";
    }
};

class C: B, A{
public:
    C():A(), B(){ // <----
        std::cout << "C";
    }
};

int main(){
    C c;    // B A C <----
}
```

В случае нескольких предков, и без явного вызова конструкторов, будут вызываться конструкторы предков в порядке их перечисления в списке наследования.

В случае нескольких предков, с явным вызовом их конструкторов, они всё равно будут вызваны в порядке их перечисления в списке наследования, а не в том, в котором они будут указаны в списке инициализаторов.

Такое же поведение в списке инициализаторов для полей класса, т.е. порядок инициализации определяется порядком перечисления полей в объявлении класса.

Конструктор при виртуальном наследовании

```
#include <iostream>
#include <string>

class Zero{
public:
    Zero(){
        std::cout << "Zero ";
    }
};

class A: Zero{
public:
    A(){
        std::cout << "A ";
    }
};

class B: Zero{
public:
    B(){
        std::cout << "B ";
    }
};

class C: B, A{
public:
    C():A(), B(){
        std::cout << "C";
    }
};

int main(){
    C c;    // Zero B Zero A C
}
```

```
#include <iostream>
#include <string>

class Zero{
public:
    Zero(){
        std::cout << "Zero ";
    }
};

class A: virtual Zero{
public:
    A(){
        std::cout << "A ";
    }
};

class B: virtual Zero{
public:
    B(){
        std::cout << "B ";
    }
};

class C: B, A{
public:
    C():A(), B(){
        std::cout << "C";
    }
};

int main(){
    C c;    // Zero B A C
}
```

```
#include <iostream>
#include <string>

class Zero{
public:
    Zero(int i){
        std::cout << i;
    }
};

class A: virtual Zero{
public:
    A(): Zero(0){
        std::cout << "A ";
    }
};

class B: virtual Zero{
public:
    B(): Zero(1){
        std::cout << "B ";
    }
};

class C: B, A{
public:
    C(): A(), B(){
        std::cout << "C";
    }
};

int main(){
    C c;    // Ошибка. У Zero нет
конструктора по умолчанию
}
```

В случае обычного ромбовидного наследования, конструктор общего предка отработает 2 раза, будучи вызванным каждым из его непосредственных наследников.

В случае виртуального ромбовидного наследования, конструктор общего предка отработает один раз, причём его вызовет последний наследник.

Конструктор при виртуальном наследовании

```
#include <iostream>
#include <string>

class Zero{
public:
    Zero(int i){
        std::cout << i;
    }
};

class A: virtual Zero{
public:
    A(): Zero(0){
        std::cout << "A ";
    }
};

class B: virtual Zero{
public:
    B(): Zero(1){
        std::cout << "B ";
    }
};

class C: B, A{
public:
    C(): Zero(10), A(), B(){
        std::cout << "C";
    }
};

int main(){
    C c;    // Ошибка. Конструктор
           // Zero не доступен из-за приватного
           // наследования
}
```

```
#include <iostream>
#include <string>

class Zero{
public:
    Zero(int i){
        std::cout << i;
    }
};

class A: virtual public Zero{
public:
    A(): Zero(0){
        std::cout << "A ";
    }
};

class B: virtual public Zero{
public:
    B(): Zero(1){
        std::cout << " B ";
    }
};

class C: B, A{
public:
    C(): Zero(10), A(), B(){
        std::cout << "C";
    }
};

int main(){
    C c;    // 10 B A C
}
```

Виртуальный конструктор

```
class A{  
public:  
    virtual A(){  
    }  
};
```

Конструктор НЕ может быть виртуальным.

Деструктор

Деструктор по умолчанию

```
class A{  
    // Присутствует реализация деструктора от компилятора  
};  
  
class B: A{  
    // Присутствует реализация деструктора от компилятора  
};  
  
int main(){  
    B b;  
}
```

Деструктор класса наследника сначала отработывает сам, а затем вызывает деструктор предка.

Если деструктор объявленный пользователем отсутствует, то вызывается деструктор по умолчанию предоставленный компилятором.

Деструктор по умолчанию

```
#include <iostream>
#include <string>

class A{
    ~A(){
        std::cout << "~A" << std::endl;
    }
};

class B: A{
    // Присутствует реализация деструктора от компилятора
};

int main(){
    B b; // Ошибка. Деструктор класса A приватный
}
```

Деструктор по умолчанию

```
#include <iostream>
#include <string>

class A{
public:
    ~A(){
        std::cout << "~A" << std::endl;
    }
};

class B: A{
    // Присутствует реализация деструктора от компилятора. Ничего не выводит
};

int main(){
    B b; // Вывод: ~A
}
```

Деструктор по умолчанию

```
#include <iostream>
#include <string>

class A{
public:
    ~A(){
        std::cout << "~A " << std::endl;
    }
};

class B: A{
public:
    ~B(){          // Вызывается деструктор по умолчанию предка
        std::cout << "~B ";
    }
};

int main(){
    B b1;          // Вывод: ~B ~A
}
```

При объявлении своего деструктора в классе наследнике, всё равно автоматически вызывается деструктор по умолчанию класса предка после него.

Явно вызвать деструктор класса предка нельзя.

Деструктор в цепочке наследования

```
#include <iostream>
#include <string>

class A{
public:
    ~A(){
        std::cout << "~A ";
    }
};

class B: A{
public:
    ~B(){
        std::cout << "~B ";
    }
};

class C: B{
public:
    ~C(){
        std::cout << "~C ";
    }
};

int main(){
    C c;    // ~C ~B ~A
}
```

В случае цепочки предков деструкторы отработают в порядке от наследника к предку и т.д.

Деструктор порядок вызова

```
#include <iostream>
#include <string>

class A{
public:
    ~A(){
        std::cout << "~A ";
    }
};

class B{
public:
    ~B(){
        std::cout << "~B ";
    }
};

class C: A, B{
public:
    ~C(){
        std::cout << "~C ";
    }
};

int main(){
    C c;    // ~C ~B ~A < ---
}
```

```
#include <iostream>
#include <string>

class A{
public:
    ~A(){
        std::cout << "~A ";
    }
};

class B{
public:
    ~B(){
        std::cout << "~B ";
    }
};

class C: B, A{
public:
    ~C(){
        std::cout << "~C ";
    }
};

int main(){
    C c;    // ~C ~A ~B < ---
}
```

В случае нескольких предков, будут вызываться деструкторы предков в **обратном** порядке их перечисления в списке наследования.

Деструктор при виртуальном наследовании

```
#include <iostream>
#include <string>

class Zero{
public:
    ~Zero(){
        std::cout << "~Zero ";
    }
};

class A: Zero{
public:
    ~A(){
        std::cout << "~A ";
    }
};

class B: Zero{
public:
    ~B(){
        std::cout << "~B ";
    }
};

class C: B, A{
public:
    ~C(){
        std::cout << "~C ";
    }
};

int main(){
    C c;    // ~C ~A ~Zero ~B ~Zero
}
```

```
#include <iostream>
#include <string>

class Zero{
public:
    ~Zero(){
        std::cout << "~Zero ";
    }
};

class A: virtual Zero{
public:
    ~A(){
        std::cout << "~A ";
    }
};

class B: virtual Zero{
public:
    ~B(){
        std::cout << "~B ";
    }
};

class C: B, A{
public:
    ~C(){
        std::cout << "~C ";
    }
};

int main(){
    C c;    // ~C ~A ~B ~Zero
}
```

В случае обычного ромбовидного наследования, деструктор общего предка отработает 2 раза, будучи вызванным каждым из его непосредственных наследников.

В случае виртуального ромбовидного наследования, деструктор общего предка отработает один раз, причём его вызовет последний наследник.

Виртуальный деструктор

```
#include <iostream>
#include <string>

class A {
public:
    ~A(){
        std::cout << "~A ";
    }
};

class B: public A{
public:
    ~B(){
        std::cout << "~B ";
    }
};

int main(){
    B b; // ~B ~A
    A* a = dynamic_cast<A*>(new B);
    delete a; // ~A
}
```

```
#include <iostream>
#include <string>

class A {
public:
    virtual ~A(){
        std::cout << "~A ";
    }
};

class B: public A{
public:
    virtual ~B(){
        std::cout << "~B ";
    }
};

int main(){
    B b; // ~B ~A
    A* a = dynamic_cast<A*>(new B);
    delete a; // Всё равно ~B ~A
}
```

Виртуальный деструктор необходим, если ваш класс владеет ресурсом, т.е. выделяет его вручную, например при помощи `new`. В противном случае классу не нужен никакой деструктор.

Виртуальный деструктор работает также как и не виртуальный. Отличие заключается в том, какой деструктор будет вызван в случае каста объекта к предку через указатель или ссылку.

Конструктор копирования,
перемещения и оператор
присваивания

Конструктор копирования

```
#include <iostream>
#include <string>

class A {
public:
    A(){
        std::cout << "A ";
    }
    A(const A& other){
        std::cout << "copy A ";
    }
};

class B: public A{
public:
    B(){
        std::cout << "B ";
    }
};

int main(){
    B b1;        // A B
    B b2(b1);    // copy A
}
```

```
#include <iostream>
#include <string>

class A {
public:
    A(){
        std::cout << "A ";
    }
    A(const A& other){
        std::cout << "copy A ";
    }
};

class B: public A{
public:
    B(){
        std::cout << "B ";
    }
    B(const B& other){
        std::cout << "copy B ";
    }
};

int main(){
    B b1;        // A B
    B b2(b1);    // A copy B
}
```

```
#include <iostream>
#include <string>

class A {
public:
    A(){
        std::cout << "A ";
    }
    A(const A& other){
        std::cout << "copy A ";
    }
};

class B: public A{
public:
    B(){
        std::cout << "B ";
    }
    B(const B& other): A(other){
        std::cout << "copy B ";
    }
};

int main(){
    B b1;        // A B
    B b2(b1);    // copy A copy B
}
```

Конструктор копирования – отвечает за корректное копирование объекта;

Конструктор копирования автоматически вызывается в тех местах, где требуется создать копию объекта: при инициализации одного объекта другим, передача в функцию по значению, возврат из функции по значению;

В случае наследования и отсутствия явно объявленного конструктора копирования, версия предоставленная компилятором вызывает **перед собой** конструкторы копирования предков, а те своих и т.д. до самого первого.

В случае наследования и явно объявленного конструктора копирования, он ведёт себя как обычный конструктор, т.е. вызывает **обычные** конструкторы по умолчанию классов предков, поэтому вызывать конструкторы копирования классов предков нужно явно.

Конструктор копирования - срезка

```
#include <iostream>
#include <string>

class A {
public:
    A(){
        std::cout << "A ";
    }
    A(const A& other){
        std::cout << "copy A ";
    }
};

class B: public A{
public:
    B(){
        std::cout << "B ";
    }
    B(const B& other): A(other){
        std::cout << "copy B ";
    }
};

void print(A obj){
}

int main(){
    B b1;        // A B
    print(b1);   // copy A <- срезка
}
```

Если передать объект наследник в функцию или вернуть из функции по значению, и при этом функция ожидает/возвращает класс предок, то это допустимый, с точки зрения синтаксиса код.

Компилятор вызовет конструктор копирования того класса который указан в параметрах или как возвращаемое значение и передаст туда объект наследник.

Конструктор копирования предка выполнит копирование только той части класса которая относится к предку (очевидно). Таким образом часть данных (если они есть) относящаяся к наследнику будет проигнорирована, иначе говоря срезана.

Срезка так же произойдёт, в случае инициализации объекта предка при помощи объекта наследника.

При передаче объекта по ссылке или по указателю никакого копирования не происходит и соответственно срезки тоже.

Конструктор перемещения

Конструктор перемещения – отвечает за корректное копирование объекта который вот-вот должен быть уничтожен (т.е. временного объекта);

Если в теле класса не объявлен конструктор перемещения, то компилятор предоставляет реализацию по умолчанию. Если объявлен конструктор копирования, то конструктор перемещения компилятором не создаётся.

Подробнее данный конструктор будет рассмотрен на следующих лекциях.

В случае наследования конструктор перемещения ведёт себя также как и конструктор копирования.

```
class Point{  
public:  
    Point(const Point& src);    // копирующий конструктор  
    Point(Point&& src) noexcept; // перемещающий конструктор  
};
```

Оператор присваивания

```
#include <iostream>
#include <string>

class A {
public:
    A(){
        std::cout << "A ";
    }
    A& operator=(const A& other){
        std::cout << "A= ";
        return *this;
    }
};

class B: public A{
public:
    B(){
        std::cout << "B ";
    }
};

int main(){
    B b1;    // A B
    B b2;    // A B
    b2 = b1; // A=
}
```

```
#include <iostream>
#include <string>

class A {
public:
    A(){
        std::cout << "A ";
    }
    A& operator=(const A& other){
        std::cout << "A= ";
        return *this;
    }
};

class B: public A{
public:
    B(){
        std::cout << "B ";
    }
    B& operator=(const B& other){
        std::cout << "B= ";
        return *this;
    }
};

int main(){
    B b1;    // A B
    B b2;    // A B
    b2 = b1; // B=
}
```

```
#include <iostream>
#include <string>

class A {
public:
    A(){
        std::cout << "A ";
    }
    A& operator=(const A& other){
        std::cout << "A= ";
        return *this;
    }
};

class B: public A{
public:
    B(){
        std::cout << "B ";
    }
    B& operator=(const B& other){
        A::operator=(other); // Вызываем у предка
        std::cout << "B= ";
        return *this;
    }
};

int main(){
    B b1;    // A B
    B b2;    // A B
    b2 = b1; // A= B=
}
```

Оператор присваивания — отвечает за корректное освобождение данных текущего объекта и копирование в него данных другого объекта в процессе присваивания;

Оператор присваивания автоматически вызывается в тех местах, где происходит присваивание (но не инициализация);

В случае наследования и отсутствия явно объявленного оператора присваивания, вызывается версия предоставленная компилятором. Она вызывает операторы присваивания предков начиная с самого дальнего предка к т.д. к наследнику, а поля текущего объекта присваивает почленно. Для простых типов побайтово, а для объектов — вызывая их операторы присваивания.

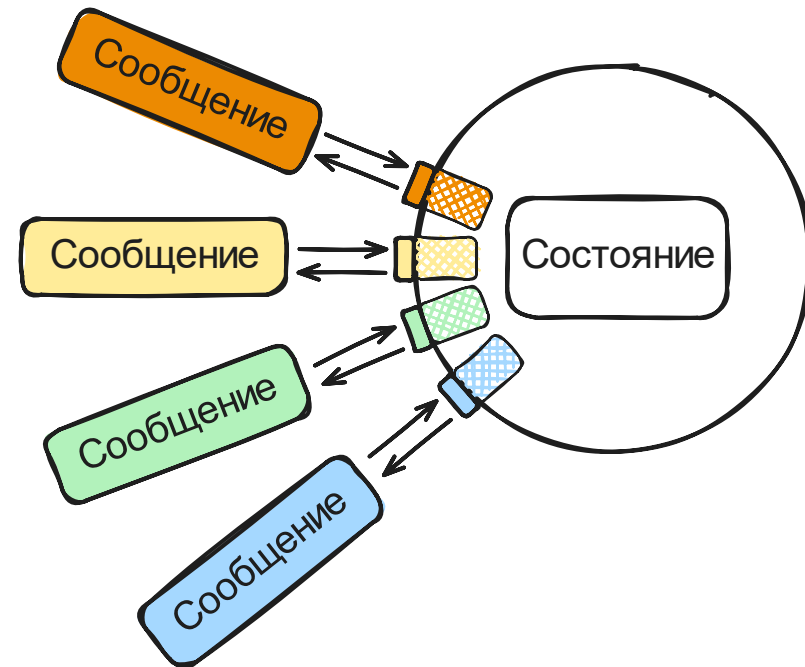
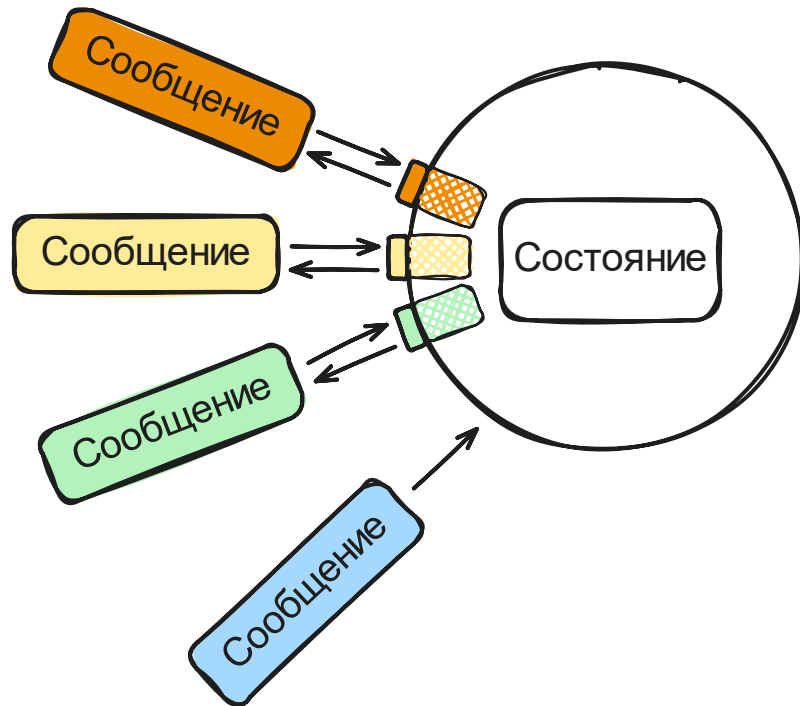
В случае наследования и явно объявленного оператора присваивания, он НЕ вызывает оператор присваивания предка и его нужно явно вызывать в коде оператора присваивания наследника в начале.

Если класс владеет данными, нужно делать виртуальным.

Подтипизация

Динамическое связывание

Идея взаимодействующих посредством сообщений объектов расширяется идеей возможности динамической замены одного объекта на другой прямо во время работы системы, при условии, что другой объект умеет обрабатывать все сообщения также как это делал первый, т.е. при условии сохранения контракта. Новый объект может быть по-другому устроен внутри или обладать дополнительным функционалом по сравнению со старым.



Контракт

При замене одного объекта на другой мы должны обеспечить исполнение Контракта.

Не считается нарушением контракта:

- Ослабление предусловий (requirements) – набор требований нового объекта может быть менее жёстким. Т.е. раньше мы могли принимать на вход только текст в формате JSON, то новый объект может **дополнительно** принимать и YAML. Т.е. менее жёсткие требования;
- Усиление постусловий (guarantees) – набор гарантий со стороны нового объекта может быть более жёстким. Т.е. ранее время исполнения метода определялось квадратичной сложностью, а теперь линейной. Т.е. метод стал работать быстрее, вкладываясь в более жёсткие рамки по времени;
- Сохранение инвариантов класса. Чтобы там ни было мы должны сохранять инварианты и проектировать контракт.

Подтипизация

В компилируемых языках часто используется статическая типизация и обычные типы, как правило не совместимы друг с другом. Переменная (или параметр функции) связывается с некоторым фиксированным типом и не может изменить его до момента своего уничтожения. Поэтому динамическое связывание реализуется через Подтипизацию т.е. построение связанной иерархии типов, которые допускают неявное преобразование друг в друга. Если объекты связаны в иерархию типов, то их можно использовать для замены друг друга.

```
obj                // Делает что-то важное в системе
obj = Type1()      // Сначала был одним объектом
obj = Type2()       // Хотим заменить на другой. Для разных типов такое запрещено
obj = SubType1()    // ОК, если типы связаны в иерархию типов
```

В интерпретируемых языках, часто используется динамическая типизация, поэтому переменная (и др.) может, во время работы программы, менять свой тип и поэтому динамическое связывание идёт здесь из коробки и не требует дополнительных телодвижений.

```
obj = Type1()      // Сначала был одним объектом
obj = Type2()       // Хотим заменить на другой. Ок, без проблем
```

Подтипизация

Встречается:

- Номинальная подтипизация, при которой только типы, объявленные определенным образом, могут быть подтипами друг друга (C++, Java);
- Структурная подтипизация, при которой структура двух типов определяет, является ли один подтип подтипом другого (Go). Это так называемая утиная типизация.

Тип данных (тип) — множество значений и операций над этими значениями [https://ru.wikipedia.org/wiki/Тип_данных].

Подтип. Если S является подтипом T , то отношение подтипов (записывается как $S <: T$ или $S \leq: T$, например Собака $<:$ Животное) означает, что любой элемент типа S может безопасно использоваться в любом контексте, где ожидается термин типа T [<https://en.wikipedia.org/wiki/Subtyping>]. Этот принцип организации подтипов был предложен Барбарой Лисков в 1987 году и дальнейшей был включён в набор принципов SOLID как принцип [подстановки Барбары Лисков](#) (LSP).

Полиморфизм

Полиморфизм – третий принцип ООП.

Если функция способна обрабатывать только один тип данных – это мономорфная функция. Например в Go нет перегрузки (про дженерики и интерфейсы пока забываем), поэтому все функции **мономорфны** (топо - один).

Полиморфизм (poly – много, morphe - форма) – возможность функции/метода обрабатывать объекты разных типов данных. Здесь важно то, как вы воспринимаете слово «функция». Его нужно понимать как в математике:

$$F(x) = \begin{cases} f(x), & x > a \\ g(x), & x \leq a \end{cases}$$

Т.е. если у вас в коде будет несколько функций, с одним и тем же именем, то смотреть на это нужно как на одну функцию, просто её поведение меняется в зависимости от количества параметров, типов и т.д. и вы прописали не разные функции, а просто поведение одной функции в зависимости от условий.

Другое определение полиморфизма – один интерфейс, множество реализаций [Б. Страуструп].

Полиморфизм

- Параметрический. В C++ реализуется при помощи шаблонов. Тип является параметром, в зависимости от которого функция обрабатывает данные по разному:

```
template<typename T>
T sum(T a, T b){
    return a + b;
}
```

```
sum<int>(10, 20);           // 30 - Функция sum складывает числа
sum<string>("10", "20");   // 1020 - Функция sum склеивает строки
```

- ad-hoc. В C++ реализуется при помощи перегрузки. "Одна и та же" функция ведёт себя по разному в зависимости от типа и/или количества аргументов (здесь нет параметра определяющего тип):

```
int sum(int a, int b, int c){ return a + b + c; }
string sum(string a, string b){ return a + b; }
```

```
sum(1, 2, 3); // 6 - Функция sum складывает числа
sum("1", "2"); // 12 - Функция sum склеивает строки
```

- Полиморфизм подтипов. В C++ реализуется посредством виртуальных функций и наследования. Объекты входящие в одну иерархию наследования можно преобразовывать друг в друга, а точнее указатели и ссылки на эти объекты. Поэтому функция будет работать по разному в зависимости от фактического типа переданного ей:

```
struct A{
    virtual void speak(){ cout << "I am A" << endl; }
};

struct B: A {
    virtual void speak(){ cout << "I am B" << endl; }
};

void greet(A& obj){ obj.speak(); }
```

```
A obj_a;
greet(obj_a); // Для типа A вывод: I am A
B obj_b;
greet(obj_b); // Для типа B поведение другое: I am B
```

Абстракция

Как правило, выстраивая иерархию подтипов руководствуются принципом – от наиболее абстрактного к более конкретному.

Абстрагирование — это такой логический приём, с помощью которого мы мысленно выделяем существенные свойства предметов, явлений и отвлекаем их от несущественных, второстепенных свойств предметов, явлений материального мира.

Результат абстрагирования называется абстракцией.

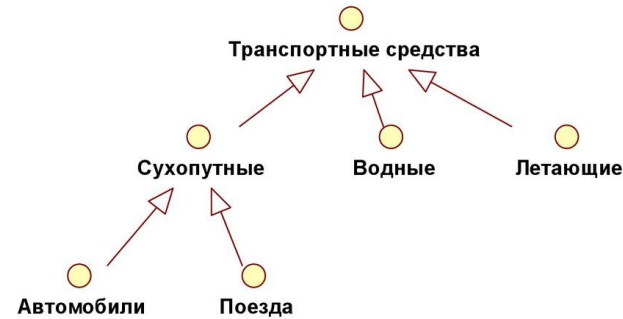
Виноградов, Логика :
Учебник логики : учебник
для сред. школы, 1954 г.

В качестве примера абстракций может быть: форма, цвет, возможность нажать (нажимаемое), пишущее и т.д. Т.е. мы выделяем нужный нам набор свойств, а остальные игнорируем.

Таким образом на вершине иерархии типов обычно находится тип с наименьшим количеством свойств, например: животное. Ниже по иерархии (наследники) типы с некоторыми доп. свойствами, например: млекопитающие, птицы, Ещё ниже типы (наследники наследников) с ещё большим количеством свойств: кошки, собаки, люди. И т.д. по мере необходимости, каждый раз переходя от более абстрактного типа к более конкретному.

Важно, чтобы в иерархии подтипов, каждый новый уровень добавлял новые свойства, но не нарушал старые. Если такое происходит, возможно вы неправильно определили уровень абстракции или типы вообще не связаны.

Абстракция



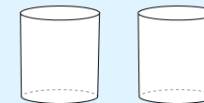
Таких котов не существует, но несмотря на это, любой ответит, что на картине изображён кот. Здесь мы обращаем внимание только на **часть** характерных **признаков** кота, а остальные игнорируем, т.е. используем абстракцию.



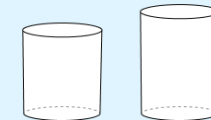
У детей способность абстрактно мыслить присутствует не с самого рождения, а развивается с возрастом.

Феномены Жана Пиаже. Перед ребёнком ставят две одинаковые прозрачные ёмкости с равным количеством воды. Взрослый спрашивает: «В какой стакан налито больше?» Ребёнок отвечает, что количество воды в обеих ёмкостях одинаковое. Взрослый берёт одну из ёмкостей и, на глазах у ребёнка, выливает из неё воду в третий стакан меньшего диаметра. В этом стакане, поскольку он более вытянутый, уровень воды выше. Ребёнок полагает, что количество воды теперь неодинаково. Раз в одном из стаканов уровень воды выше — значит, там её больше.

Сохранение количества жидкости



Ребёнку показывают два одинаковых стакана с равными уровнями жидкости



Экспериментатор переливает жидкость из одного стакана в другой, более высокий и узкий

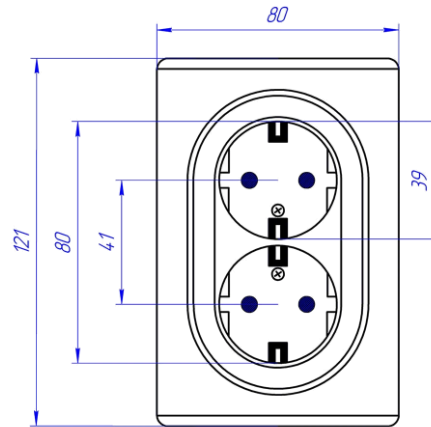
Интерфейс

Интерфейс – это часть контракта определяющая способ взаимодействия систем друг с другом.

Функция – это код расположенный где-то в памяти компьютера. Прототип/сигнатура функции – это интерфейс определяющий способ обратиться к этому фрагменту кода, передать ему данные и получить данные в ответ. Прототип не описывает ожидание функции от данных или гарантии результата:

```
int findIndex(vector<int>& array, int value); // А если не найден?
```

Электрическая сеть дома позволяет получить электроэнергию. Розетка – это интерфейс, определяющий способ подключения к электрической сети для электроприборов. Розетка не описывает частоту, напряжение, предельную мощность и другие моменты входящие в состав контракта взаимодействия с электрической сетью.



Интерфейс класса

Интерфейсом класса обычно называют список его публичных методов. Но если вы разместите поле в публичной секции, то формально оно тоже станет частью интерфейса.

Если разобраться ещё более детально, то методы и поля размещённые в секции `protected` также можно отнести к интерфейсу класса. Хотя они и не доступны всем, но всё-таки позволяют наследникам вмешиваться в работу класса и рассчитывать на то, что эти методы не будут изменены в будущем.

Дружественные функции – также можно назвать частью интерфейса класса.

Обычной практикой является стремление к уменьшению интерфейса на сколько это возможно. В идеале интерфейс должен состоять из одного метода.

Например в Java:

```
interface Runnable{  
    void run();  
}
```

Интерфейс класса

В C++ отсутствует специальное ключевое слово для обозначения интерфейсов, поэтому эквивалентом можно назвать классы содержащие только публичные чистые виртуальные методы:

C++

```
class Runnable{  
public:  
    virtual void run() = 0;  
}
```

Java

```
interface Runnable{  
    void run();  
}
```

Класс у которого есть хотя бы один чистый виртуальный метод можно назвать абстрактным классом. Экземпляр такого класса создать нельзя. От него можно только наследоваться, и если определить реализацию для всех чистых виртуальных методов, то экземпляр создать можно будет. Обычно, абстрактные классы служат для предоставления базового функционала для классов-наследников, а они в свою очередь переопределяют нужные методы.

Наследование реализации

В C++ нет возможность наследования реализации без наследования интерфейса. Это можно сделать при помощи обходного пути:

```
class A{
public:
    void print(){
        std::cout << "+" << std::endl;
    }
};

class B: private A{
public:
    void plus(){ // По сути тот же print только под другим именем
        print(); // или A::print();
    }
};

B object;
object.print(); // Ошибка
object.plus();  // ОК
```

Наследование интерфейса и реализации

Обычно интерфейс наследуется вместе с реализацией:

```
class A{  
public:  
    void print(){  
        std::cout << "+" << std::endl;  
    }  
};
```

```
class B: public A{  
};
```

```
B object;  
object.print();    // OK
```

При обычном наследовании, без намеренного построения иерархии подтипов, к унаследованному интерфейсу стоит относиться как к просто похожему.



Создание иерархии типов через наследование

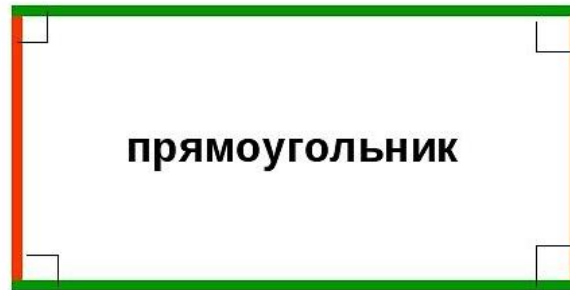
При целенаправленном построении иерархии типов следует пользоваться двумя проверочными правилами: is-a (является) и правилом подстановки Барбары Лисков.

Если при подстановке в выражение: «класс-наследник *является* классом-предком», вы получаете явное противоречие, то классы точно не следует объединять в иерархию типов. Например: кот является коробкой; вода является кувшином. Здесь понятно, что кот не является подтипом коробки, а вода не является подтипом кувшина, поэтому с точки зрения подтипизации неправильно наследовать кота от коробки, и воду от кувшина.

Если при подстановке, вы получаете осмысленное выражение, то есть вероятность, что классы являются подтипами. В этом случае следует воспользоваться правилом подстановки Барбары Лисков. В одной из формулировок оно звучит как: "Функции, которые используют базовый тип, должны иметь возможность использовать подтипы базового типа, не зная об этом".

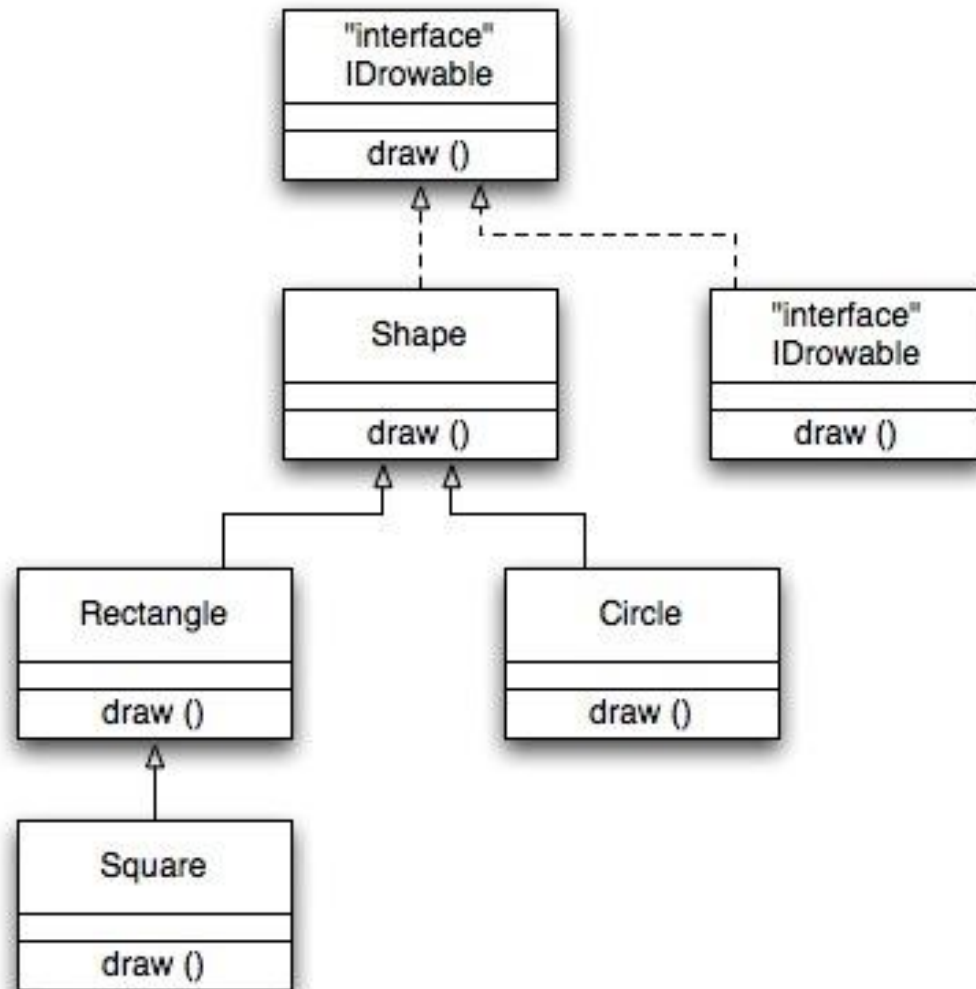
Нужно понимать, что однозначного ответа что правильно, а что нет – не существует(. Ситуация полностью определяется **контрактом**, который вы определите. Именно поэтому очень важно его чётко прописать.

Кто кому предок?



Эквивалентная задача: задача о круге и эллипсе.

Как вариант?



Квадрат подтип прямоугольника

С точки зрения математики, квадрат – это прямоугольник у которого высота и ширина равны друг другу. Как видно, правило *is-a* соблюдено.

Распишем контракты для обоих классов:

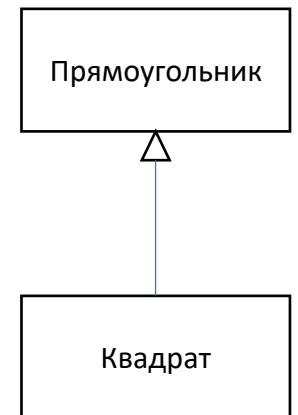
```
class Rectangle{ // Прямоугольник
protected:
    int width; // Ширина
    int height; // Высота

public:
    Rectangle(int w, int h): width(w), height(h) {}

    virtual int getArea(){
        // Контракт: определяет площадь фигуры
        // Требования: ширина и высота > 0
        // Гарантии: результат > 0
        return width * height;
    }
};
```

```
class Square: public Rectangle{ // Квадрат
// Инвариант: ширина и высота равны друг другу
public:
    Square(int side): Rectangle(side, side) {}

    virtual int getArea(){
        // Контракт: определяет площадь фигуры
        // Требования: ширина и высота равны и > 0
        // Гарантии: результат > 0
        return width * height;
    }
};
```



Как видно, требования в `getArea` квадрата усилены, что является нарушением контракта. В данном конкретном случае, мы можем это пропустить. Если класс прямоугольник не планируется расширять, то **такая система типов допустима**. Даже если мы определим метод расчёта площади квадрата через `width * width`, здесь всё будет ОК.

Квадрат подтип прямоугольника

```
class Rectangle{ // Прямоугольник
protected:
    int width; // Ширина
    int height; // Высота

public:
    Rectangle(int w, int h): width(w), height(h) {}

    virtual int getArea(){
        // Контракт: определяет площадь фигуры
        // Требования: ширина и высота > 0
        // Гарантии: результат > 0
        return width * height;
    }

    virtual void setWidth(int w){
        // Контракт: изменяет ширину фигуры
        // Требования: новая ширина > 0
        // Гарантии: изменяется только ширина фигуры
        width = w;
    }
};
```

```
class Square: public Rectangle{ // Квадрат
// Инвариант: ширина и высота равны друг другу
public:
    Square(int side): Rectangle(side, side) {}

    virtual int getArea(){
        // Контракт: определяет площадь фигуры
        // Требования: ширина и высота равны и > 0
        // Гарантии: результат > 0
        return width * height;
    }

    virtual void setWidth(int w){
        // Контракт: изменяет ширину и высоту фигуры
        // Требования: новая ширина > 0
        // Гарантии: изменяется ширина и высота фигуры
        width = w;
        height = w;
    }
};
```

Если мы добавим в класс предок метод изменяющий ширину `setWidth`, то метод наследника будет явно нарушать контракт ослабляя гарантии метода предка. Таким образом, если будет функция `f` ожидающая на вход `Rectangle` и изменяющая его ширину, и она получит на вход `Square`, то результат будет непредсказуем, т.к. мы нарушаем гарантии обещанные `Rectangle`, а код функции `f` может на эти гарантии рассчитывать. **Т.е. объединять их в одну иерархию таким образом – сомнительное решение.**

Прямоугольник подтип квадрата

Прямоугольник – это квадрат у которого ширина и высота различны. Как видно, правило *is-a* уже звучит не очень хорошо.

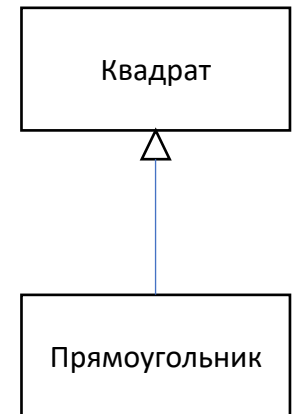
Распишем контракты для обоих классов:

```
class Square{    // Квадрат
protected:
    int side;    // Размер стороны
// Инвариант: ширина и высота равны друг другу
public:
    Square(int s): side(s) {}

    virtual int getArea(){
        // Контракт: определяет площадь фигуры
        // Требования: ширина и высота равны и > 0
        // Гарантии: результат > 0
        return side * side;
    }
};
```

```
class Rectangle: public Square{ // Прямоугольник
protected:
    // side    // Ширина
    int side2; // Высота
public:
    Rectangle(int w, int h): Square(w), side2(h) {}

    virtual int getArea(){
        // Контракт: определяет площадь фигуры
        // Требования: ширина и высота > 0
        // Гарантии: результат > 0
        return side * side2;
    }
};
```



Как видно, код тоже получился странноват, по крайней мере с точки зрения имён полей. Если посмотреть на метод `getArea`, то мы видим, что контракт не нарушается, т.к. предусловие ослаблено (стороны могут быть и равны и не равны). Но здесь есть проблема посерьёзнее. Конструктор класса `Rectangle` позволяет создавать объекты нарушающие инвариант `Square`. Если класс квадрат не планируется расширять, то **такая система типов допустима**.

Прямоугольник подтип квадрата

```
class Square{    // Квадрат
protected:
    int side;    // Размер стороны
// Инвариант: ширина и высота равны друг другу
public:
    Square(int s): side(s) {}

    virtual int getArea(){
        // Контракт: определяет площадь фигуры
        // Требования: ширина и высота равны и > 0
        // Гарантии: результат > 0
        return side * side;
    }

    friend bool operator>(const Square& lhs, const Square& rhs);    };

bool operator>(const Square& lhs, const Square& rhs){
    return lhs.side > rhs.side;
}
```

```
class Rectangle: public Square{    // Прямоугольник
protected:
    // side    // Ширина
    int side2;    // Высота

public:
    Rectangle(int w, int h): Square(w), side2(h) {}

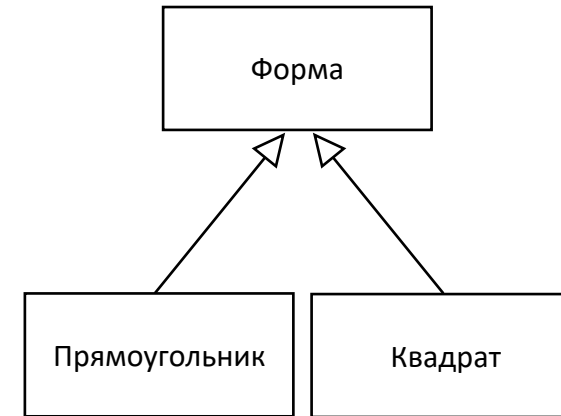
    virtual int getArea(){
        // Контракт: определяет площадь фигуры
        // Требования: ширина и высота > 0
        // Гарантии: результат > 0
        return side * side2;
    }
}
```

Здесь при подстановке прямоугольника в оператор > будет выполнено сравнение только по ширине, игнорируя высоту прямоугольника, в результате мы получим неправильно сравнение. **Т.е. объединять их в одну иерархию таким образом – сомнительное решение.**

```
Rectangle r1(10, 20), r2(10, 5);
cout << (r1 > r2); // 0
```

Квадрат прямоугольник классы одного уровня

```
class Shape{ // Интерфейсный класс Форма
public:
    virtual int getArea() = 0; // Чистый виртуальный метод
    // Контракт: определяет площадь фигуры
    // Требования: -
    // Гарантии: результат > 0
};
```



```
class Square: public Shape{ // Квадрат
    int side; // Размер стороны
    // Инвариант: ширина и высота равны друг другу и равны side
public:
    Square(int s): side(s) {}
    virtual int getArea(){
        // Контракт: определяет площадь фигуры
        // Требования: ширина и высота равны и > 0
        // Гарантии: результат > 0
        return side * side;
    }
};
```

```
class Rectangle: public Shape{ // Прямоугольник
    int width; // Ширина
    int height; // Высота
public:
    Rectangle(int w, int h): width(w), height(h) {}
    virtual int getArea(){
        // Контракт: определяет площадь фигуры
        // Требования: ширина и высота > 0
        // Гарантии: результат > 0
        return width * height;
    }
};
```

Пример – ослабление требований

```
class PaymentProcessor {
public:
    virtual bool process(double amount) {
        // Предусловие: amount > 0

        if (amount <= 0) {
            throw std::invalid_argument("Сумма должна быть положительной");
        }
        // Выполняем обработку
        return true;
    }
};
```

```
class DiscountPaymentProcessor: public PaymentProcessor {
public:
    bool process(double amount) override {
        // Предусловие ослаблено: amount >= 0

        if (amount < 0) { // Теперь 0 тоже допустимое количество
            throw std::invalid_argument("Сумма не может быть отрицательной");
        }
        // Выполняем обработку
        return true;
    }
};
```

Наследник может обрабатывать как все запросы, которые может обрабатывать предок, так и дополнительный вариант при котором количество равно нулю, поэтому требования к вводу ослаблены и контракт не нарушен.

Пример – ослабление требований

```
class FileUploader {
public:
    virtual void upload(const string& filename, long fileSize) {
        // Исходные требования:
        // 1. Имя файла не пустое
        // 2. Размер > 0 и ≤ 10MB

        if (filename.empty()) {
            throw invalid_argument("Filename cannot be empty");
        }

        if (fileSize <= 0) {
            throw invalid_argument("File must have positive size");
        }

        if (fileSize > 10 * 1024 * 1024) {
            throw invalid_argument("File too large (max 10MB)");
        }

        cout << "Uploading file: " << filename << endl;
    }
};
```

```
class PremiumFileUploader : public FileUploader {
public:
    void upload(const string& filename, long fileSize) override {
        // Ослабление требований:
        // 1. Имя файла может быть пустым (автогенерация)
        // 2. Размер > 0 и ≤ 50MB

        string actualName = filename.empty()
            ? "auto_generated_" + to_string(rand())
            : filename;

        if (fileSize <= 0) {
            throw invalid_argument("File must have positive size");
        }

        if (fileSize > 50 * 1024 * 1024) {
            throw invalid_argument("File too large (max 50MB)");
        }

        cout << "Uploading premium file: " << actualName << endl;
    }
};
```

Наследник может обрабатывать как файлы без имён, так и файлы большего размера. Тут нужно быть внимательным. Исключение при пустом имени может быть как частью требования, так и частью гарантий. Зависит от вас, но это должно быть чётко прописано.

Пример – усиления гарантий

```
class Animal {
public:
    // Контракт: Возвращает звук животного
    // Гарантии: Возвращает строку - звук какого-либо животного
    virtual std::string say() const = 0;
};

class Dog: public Animal {
public:
    // Гарантии усилены: теперь звук конкретного животного "Woof"
    std::string say() const override {
        return "Woof";
    }
};
```

```
class Cat: public Animal {
public:
    // Гарантии усилены: теперь звук конкретного животного "Meow"
    std::string say() const override {
        return "Meow";
    }
};

class Fox: public Animal {
public:
    // Гарантии ОСЛАБЛЕНЫ: теперь это не звук животного
    std::string say() const override {
        return "???"
    }
};
```

Наследники Dog и Cat усиливают гарантии, т.к. гарантия базового класса Animal – это звук который издаёт любое животное, а у них конкретный звук принадлежащий этому множеству звуков. Контракт не нарушен.

Класс Fox ослабляет гарантии, т.к. возвращает символы не принадлежащие множеству звуков издаваемых животными, тем самым расширяя исходное множество и нарушая контракт.

Пример – усиления гарантий

```
class NumberGenerator {
public:
    // Контракт: Генерирует случайное число
    // Гарантии: число в диапазоне [0, 100]
    virtual int generate() const {
        static std::mt19937 gen(std::random_device{}());
        std::uniform_int_distribution<> dist(0, 100);
        return dist(gen);
    }
};

class EvenNumberGenerator : public NumberGenerator {
public:
    // Гарантии усилены: возвращает четное число в диапазоне [0, 100]
    int generate() const override {
        int number;
        do {
            number = NumberGenerator::generate(); // Используем базовую реализацию
        } while (number % 2 != 0); // Гарантируем четность

        return number; // Всегда четное число в [0, 100]
    }
};
```

```
class PrimeNumberGenerator : public NumberGenerator {
public:
    // Гарантии усилены: возвращает только простые числа в диапазоне [0, 100]
    int generate() const override {
        int number;
        do {
            number = NumberGenerator::generate(); // Используем базовую реализацию
        } while (!isPrime(number));

        return number; // Всегда простое число в [0, 100]
    }
};
```

Наследники `EvenNumberGenerator` и `PrimeNumberGenerator` усиливают гарантии, т.к. возвращают результат в диапазоне гарантированном классом `NumberGenerator`, но при этом на результат накладывается дополнительные ограничения.

Вызов метода статического типа

Как мы видели ранее преобразование/копирование/присваивание:

- объекта-наследника к объекту-предку вызывает эффект срезки (формально преобразование не вызывает срезку, но эффект похожий);
- ссылки на объект-наследник к ссылке на объект-предок приводит к тому, что вызываются методы предка, т.е. статического типа к которому кастуем;
- указателя на объект-наследник к указателю на объект-предок приводит к тому, что вызываются методы предка, т.е. статического типа к которому кастуем;

Кроме всего прочего это может вызвать проблемы, в случае если объект владеет ресурсом. В данном случае будет вызван деструктор класса A и как следствие ресурс `res` не будет освобождён, что приведёт к утечке памяти.

```
class A{
public:
    void print(){
        std::cout << "A" << std::endl;
    }
};

class B: public A{
    int* res;
public:
    B(int value): res(new int(value)){}
    ~B(){ delete res; }

    void print(){
        std::cout << "B" << std::endl;
    }
};

B obj_b;
static_cast<A>(obj_b).print();    // A
static_cast<A*>(obj_b).print();   // A
static_cast<A*>(&obj_b)->print(); // A
```

```
A* obj_b = new B(10);
delete obj_b; // Вызывается деструктор A
```

Вызов метода динамического типа

Если метод класса наследника НЕ отмечен ключевым методом `virtual`, и мы получаем к нему доступ через указатель/ссылку на класс предка, то реализация метода будет взята у предка. Что в коде написано такой и класс:

```
B* b = new B;
A* a = b;
b->print(); // B
a->print(); // A
```

Если метод класса отмечен ключевым методом `virtual`, и мы получаем к нему доступ через указатель/ссылку на класс предка, то реализация метода будет определена при помощи таблицы виртуальных методов. Виртуальная таблица методов всегда указывает на методы **реального** объекта, независимо от того через какой указатель/ссылку мы получаем к нему доступ:

```
B* b = new B;
A* a = b;
b->print(); // B
a->print(); // B
```

Поиск имён для виртуальных методов происходит также как и для не виртуальных.

```
class A{
public:
    virtual ~A() = default;

    virtual void print(){
        std::cout << "A" << std::endl;
    }
};

class B: public A{
    int* res;
public:
    B(int value): res(new int(value)){}
    virtual ~B(){ delete res; }

    virtual void print(){
        std::cout << "B" << std::endl;
    }
};

A* obj_b = new B();
obj_b->print(); // B
delete obj_b;   // Вызывается деструктор B
```

Вызов метода динамического типа

Тут есть нюанс. Метод может быть в предках НЕ виртуальный, а в наследниках стать виртуальным. Перестать быть виртуальным уже нельзя.

Если наследник будет преобразован к указателю/ссылке на предка, у которого метод ещё не был виртуальным, то он будет подчиняться правилам НЕ виртуальных методов. Поэтому не рекомендуется делать виртуальными уже существующие НЕ виртуальные методы.

Т.к. метод НЕ может перестать быть виртуальным после того как в любом предке был отмечен ключевым словом `virtual`, то в наследниках слово `virtual` можно не писать, метод всё равно будет виртуальным.

```
class A{
public:
    void print(){
        std::cout << "A" << std::endl;
    }
};

class B: public A{
public:
    virtual void print(){
        std::cout << "B" << std::endl;
    }
};

class C: public B{
public:
    virtual void print(){
        std::cout << "C" << std::endl;
    }
};

C* c = new C;
B* b = c;
A* a = c;
c->print(); // C
b->print(); // C
a->print(); // A
```

Таблица виртуальных методов

```
#include <iostream>
#include <string>

class A{
public:
    void print(){}
};

class vA{
public:
    virtual void print(){}
};

class vB: vA{
public:
    void print() override{}
};

int main(){
    std::cout << sizeof(A) << std::endl; // 1
    std::cout << sizeof(vA) << std::endl; // 8
    std::cout << sizeof(vB) << std::endl; // 8
}
```

Для обеспечения функционала виртуальных методов, в объект, класс который содержит хотя бы один виртуальный метод добавляется указатель на таблицу виртуальных методов.

Эта таблица создаётся для класса и содержит список всех виртуальных методов класса. Каждый объекта этого класса содержит указатель на эту таблицу.

В процессе вызова метода, компилятор переходит в эту таблицу и ищет запрашиваемый метод, если он там есть, то компилятор берёт адрес метода из таблицы и идёт его исполнять.

Если метода в таблице нет, то компилятор "смотрит в код" на тип указателя/ссылки и берёт метод из этого класса.

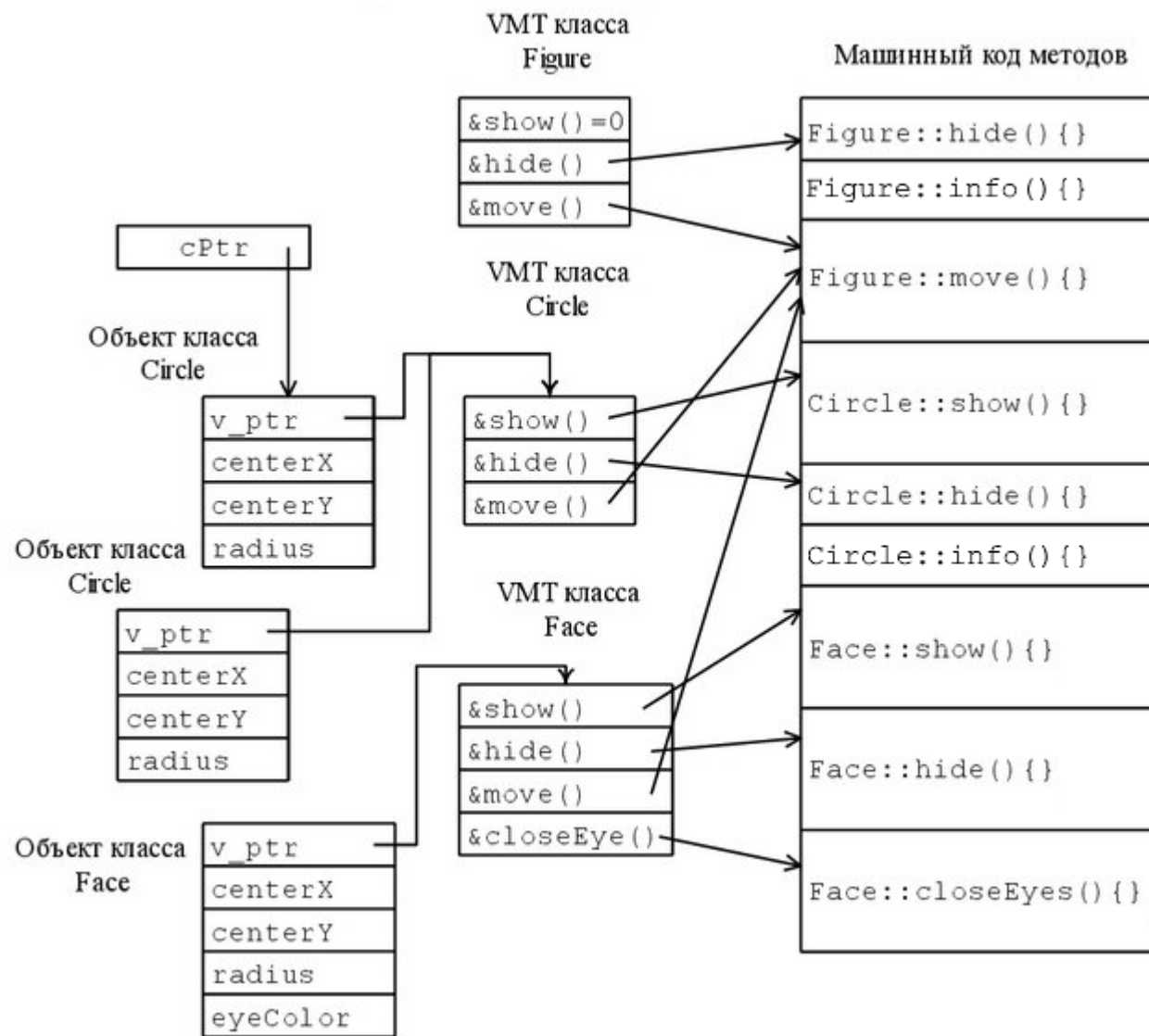
Таблица виртуальных методов

```
class Figure{
public:
    void info() {};
    virtual void show() = 0;
    virtual void hide(){
    }
    virtual void move(int x, int y){
    }
};
```

```
class Circle: public Figure{
    int centerX;
    int centerY;
    int radius;
public:
    void info() {};
    void show() override{
    }
    void hide() override{
    }
};
```

```
class Face: public Circle{
    int eyeColor;
public:
    void show() override{
    }
    void hide() override{
    }
    virtual void closeEye(bool close){
    }
};
```

```
Circle c1, c2;
Face f1;
Figure* cPtr = &c1;
cPtr->show(); // Виртуальный. Смотрим по таблице vtable
cPtr->info(); // Не виртуальный. Смотрим на тип указателя в коде
```



Проверка переопределения

Т.к. в C++ разрешена перегрузка метода; возможны опечатки, со стороны программиста; или вы просто переименовали метод, но не везде и т.д. вы можете получить НЕ переопределение метода, а его перегрузку или просто новый метод с похожим именем.

Язык предоставляет возможность проверить, что в предке действительно присутствует переопределяемый метод и он виртуальный.

Если после имени виртуального метода указать ключевое слово **override**, то, если у предка нет такого метода или он не виртуальный, будет получена ошибка компиляции.

Присутствие **override** не является обязательным, но считается хорошим тоном, как с точки зрения безопасности, так и с точки зрения обозначения, что данный метод является виртуальным, т.к. ключевое слово **virtual** обычно не пишут в наследниках.

```
class A{
public:
    virtual void print(){
        std::cout << "A" << std::endl;
    }
    void info(){
        std::cout << "A" << std::endl;
    }
};

class B1: public A{
public:
    void print() override{
        std::cout << "B1" << std::endl;
    }
};

class B2: public A{
public:
    void Print() override{ // Ошибка
        std::cout << "B2" << std::endl;
    }
    void info() override{ // Ошибка
        std::cout << "B2" << std::endl;
    }
};
```

Запрет переопределения

Виртуальный метод можно отметить ключевым словом `final`, в этом случае наследоваться от класса по прежнему можно, но переопределять метод нельзя.

С НЕ виртуальными методами `final` использовать нельзя.

```
class A{
public:
    virtual void print(){
        std::cout << "A" << std::endl;
    }
    void info() final{ // Ошибка
        std::cout << "A" << std::endl;
    }
};

class B: public A{
public:
    void print() final{
        std::cout << "B1" << std::endl;
    }
};

class C: public B{
public:
    void print() override{ // Ошибка
        std::cout << "C" << std::endl;
    }
};
```

Ковариантность возвращаемых типов

```
class A{
public:
    virtual int* get(){
        return new int;
    }
};

class B: public A{
public:
    // Ошибка. Типы не ковариантны
    double* get() override{
        return new double;
    }
};
```

```
class A{
public:
    virtual void get(A* a){
    }
};

class B: public A{
public:
    // Ошибка
    void get(B* b) override {
    }
};
```

```
class A{
public:
    virtual A* get(){
        return new A;
    }
};

class B: public A{
public:
    // OK. Типы ковариантны
    B* get() override {
        return new B;
    }
};
```

```
class A{
public:
    virtual std::vector<A*> get(){
        return {new A};
    }
};

class B: public A{
public:
    // Ошибка. Типы не ковариантны
    std::vector<B*> get() override {
        return {new B};
    }
};
```

Преобразование объектов в цепочке наследования

```
#include <iostream>
#include <string>

class A{
public:
    virtual void print(){
        std::cout << "A" << std::endl;
    }
};

class B: public A{
public:
    void print() override{
        std::cout << "B" << std::endl;
    }
};

int main(){
    B obj_b;
    static_cast<A>(obj_b).print();    // A
    static_cast<A&>(obj_b).print();   // B
    static_cast<A*>(&obj_b)->print(); // B

    dynamic_cast<A&>(obj_b).print();  // B

    A obj_a;
    static_cast<B*>(&obj_a)->print(); // A
    std::cout << (dynamic_cast<B*>(&obj_a) == nullptr); // 1
}
```

Преобразование объектов в цепочке наследования вверх по иерархии от наследника к любому предку безопасно, хотя и не всегда имеет смысл. Его называют UpCast.

Преобразование объектов в цепочке наследования вниз по иерархии от предка к наследнику как правило опасно. Его называют DownCast.

Если в классе есть виртуальные методы, нужно использовать `dynamic_cast`. Если выполнять преобразование указателей, то `dynamic_cast` выдаст нулевой указатель в случае, если преобразование некорректно, т.е. фактический тип объекта не совпадает с целевым. `static_cast` при этом выполнит преобразование, главное чтобы типы находятся в одной иерархии.

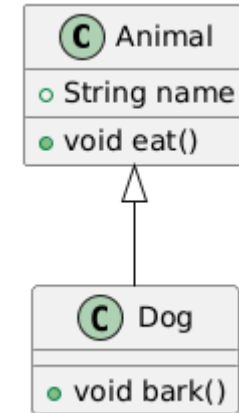
Агрегаций и композиция

Переиспользование функционала класса

Одной из причин наследования является желание переиспользование функционала существующего класса. Наследование — это не единственный способ достичь поставленной цели:

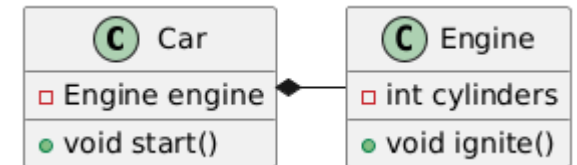
Наследование (Inheritance) — отношение «является» (is-a). Класс-потомок наследует структуру и поведение родительского класса и расширяет или изменяет его.

Пример: Dog наследует от Animal. Собака является животным.



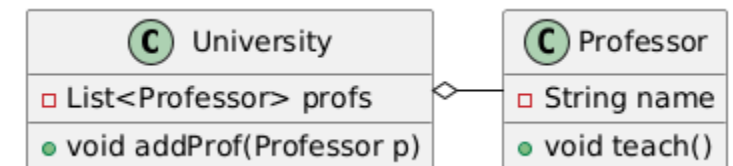
Композиция (Composition) — отношение «имеет» (has-a) с жестким контролем жизненного цикла. Объект-часть не может существовать без объекта-целого.

Пример: У Car есть Engine. Двигатель создается и уничтожается вместе с автомобилем. Без автомобиля двигатель в этом контексте не существует.



Агрегация (Aggregation) — более слабая форма отношения «имеет». Объект-часть может существовать независимо от объекта-целого и обычно передается ему извне.

Пример: В University работает Professor. Профессор существует независимо от университета и может работать в другом месте.



Наследование

```
class Animal {
protected:
    std::string name;
    int age;

public:
    Animal(const std::string& name, int age) : name(name), age(age) {
        std::cout << "Animal " << name << " created (age: " << age << ")\n";
    }

    // Виртуальный метод
    virtual void makeSound() const {
        std::cout << name << " makes a generic animal sound\n";
    }

    // Обычный метод
    void eat() const {
        std::cout << name << " is eating\n";
    }

    void sleep() const {
        std::cout << name << " is sleeping\n";
    }

    // Виртуальный деструктор – здесь не обязательно, чисто чтобы был
    virtual ~Animal() {
        std::cout << "Animal " << name << " destroyed\n";
    }

    // Геттер
    std::string getName() const { return name; }
};

// Создаем объект
Dog myDog("Buddy", 3, "Golden Retriever", true);
```

```
class Dog : public Animal {
private:
    std::string breed;
    bool isTrained;

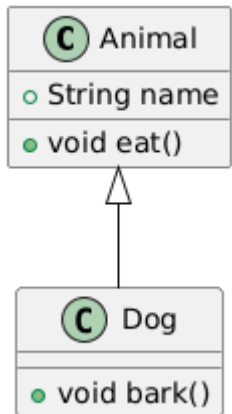
public:
    // Конструктор Dog вызывает конструктор Animal
    Dog(const std::string& name, int age, const std::string& breed, bool trained)
        : Animal(name, age), breed(breed), isTrained(trained) {
        std::cout << "Dog " << name << " created, breed: " << breed << "\n";
    }

    // Переопределяем звук
    void makeSound() const override {
        std::cout << name << " says: Woof! Woof!\n";
    }

    // Специфичный для Dog метод
    void fetch(const std::string& item) {
        std::cout << name << " is fetching " << item << "\n";
    }

    void wagTail() {
        std::cout << name << " is wagging its tail happily!\n";
    }

    // Переопределение деструктора (не обязательно, но показательно)
    ~Dog() override {
        std::cout << "Dog " << name << " destroyed\n";
    }
};
```



Композиция

```
class Car {
private:
    std::string model;
    int year;
    Engine engine; // Композиция по значению

public:
    // Конструктор инициализирует все части
    Car(const std::string& model, int year, int engineHP)
        : model(model), year(year),
          engine(engineHP, "gasoline"), // Engine создается здесь
        {} {
        std::cout << "Car " << model << " " << year << " created\n";
    }

    void start() {
        std::cout << "Starting " << model << "... \n";
        engine.start();
    }

    void drive() {
        std::cout << model << " is driving with "
                  << engine.getHorsepower() << " HP\n";
    }

    // Деструктор: все части уничтожаются автоматически
    ~Car() {
        std::cout << "Car " << model << " destroyed\n";
        // Engine уничтожается автоматически здесь
    }

    // НЕТ методов для передачи Engine извне!
    // Он создаётся и управляется только внутри Car
};
```

```
// Создаем объект
Car myCar("Toyota Camry", 2023, 203);
```

```
class Engine {
private:
    int horsepower;
    std::string fuelType;
    bool isRunning;
```

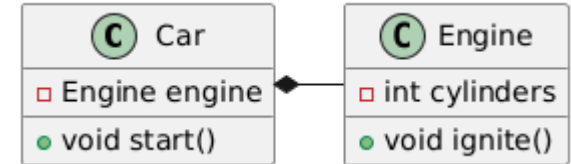
```
public:
    Engine(int hp, const std::string& fuel)
        : horsepower(hp), fuelType(fuel), isRunning(false) {
        std::cout << "Engine created: " << horsepower << " HP, " << fuelType << "\n";
    }

    void start() {
        if (!isRunning) {
            isRunning = true;
            std::cout << "Engine started. Vroom!\n";
        } else {
            std::cout << "Engine is already running\n";
        }
    }

    void stop() {
        if (isRunning) {
            isRunning = false;
            std::cout << "Engine stopped\n";
        }
    }

    int getHorsepower() const { return horsepower; }

    ~Engine() {
        std::cout << "Engine destroyed\n";
    }
};
```



Агрегация

```
class University {
private:
    std::string name;
    std::string location;

    // Агрегация: указатели/ссылки на существующие объекты
    std::vector<Professor*> professors;
public:
    University(const std::string& name, const std::string& location)
        : name(name), location(location) {
        std::cout << "University " << name << " established in " << location << "\n";
    }

    // Метод для добавления существующих объектов
    void hireProfessor(Professor* prof) {
        if (prof) {
            professors.push_back(prof);
            std::cout << prof->getName() << " hired by " << name << "\n";
        }
    }

    void displayInfo() const {
        std::cout << "\n=== " << name << " Information ===\n";
        std::cout << "Location: " << location << "\n";
        std::cout << "Professors (" << professors.size() << "):\n";
        for (auto prof : professors) {
            std::cout << "  - " << prof->getName() << " (" << prof->getSpecialty() <<
        }
        std::cout << "\n";
    }

    // ВАЖНО: деструктор НЕ удаляет агрегированные объекты, т.к. не владеет ими!
    ~University() {
        std::cout << "University " << name << " closed\n";
        // НЕ делаем delete для professors!
        // Они управляются кем-то другим
    }
};
```

```
class Professor {
private:
    std::string name;
    std::string specialty;
    int yearsExperience;
public:
    Professor(const std::string& name, const std::string& specialty, int years)
        : name(name), specialty(specialty), yearsExperience(years) {
        std::cout << "Professor " << name << " created\n";
    }

    void teach(const std::string& course) const {
        std::cout << name << " is teaching " << course << "\n";
    }

    void doResearch() const {
        std::cout << name << " is conducting research in " << specialty << "\n";
    }

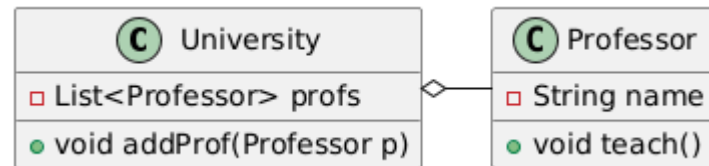
    std::string getName() const { return name; }
    std::string getSpecialty() const { return specialty; }

    ~Professor() {
        std::cout << "Professor " << name << " destroyed\n";
    }
};

// Создаем независимые объекты (части)
Professor prof1("Dr. Smith", "Computer Science", 15);
Professor prof2("Dr. Johnson", "Mathematics", 20);

// Создаем университет (целое)
University mit("MIT", "Cambridge, MA");

// Агрегируем существующие объекты
mit.hireProfessor(&prof1);
mit.hireProfessor(&prof2);
```



Сравнение

	Наследование	Композиция	Агрегация
Отношение	"является" (is-a)	"имеет" (has-a) с владением	"имеет" (has-a) без владения
Сила связи	☆☆☆☆☆ Очень высокая	☆☆☆☆ Высокая	☆☆ Средняя/Низкая
Возможность модификации	Ограниченная (через переопределение)	Полная (через замену компонента)	Полная (динамическая замена)
Связь	В надстройку поверх А	А содержит В как член	А ссылается на В
Время жизни	Совпадает	Часть создается/уничтожается с целым	Независимое
Владение	Не применимо	Строгое владение	Нет владения
Получение	Не требуется	Создается внутри	Передается извне
Отсутствие	Невозможно	Невозможно	Возможно (указатели)
Полиморфизм	Да (виртуальные функции)	Нет (если не через интерфейсы)	Нет (если не через интерфейсы)
Тестируемость	Сложная (жёсткая связь)	Средняя (можно mock через интерфейсы)	Высокая (легко подменять зависимости)